

1 How to use

Below you will find an overview with all (important) EMerge functions. Click on the function/method name and be transported to the description/documentation later. Above in the header you can find red bold letters for each category of functions (**Geometry** | **Messhing** | etc...). You can click on those words to be transported back to the these pages.

Good look and happy simulating!

Robert Fennis

2 Global Functions

2.1 Global States and Definitions

Module/Class	Function/Method	Page
em	CENTER	6
em	CORNER	6
em	EMSolver	6
em	GCS	6
em	cleanup	6

2.2 Simulation Class

Module/Class	Function/Method	Page
em	Simulation	6
em.Simulation	activate	7
em.Simulation	all_boundaries	8
em.Simulation	all_geos	8
em.Simulation	cache_build	7
em.Simulation	cache_run	7
em.Simulation	check_version	7
em.Simulation	commit_geometry	8
em.Simulation	export	8
em.Simulation	find_geo	8
em.Simulation	from_simulations	8
em.Simulation	generate_mesh	8
em.Simulation	load	7
em.Simulation	parameter_sweep	8
em.Simulation	ping	6
em.Simulation	quick_mesh	8
em.Simulation	reset	6
em.Simulation	save	7
em.Simulation	set_loglevel	7
em.Simulation	set_periodic_cell	7
em.Simulation	set_physics	7
em.Simulation	set_solver	8
em.Simulation	view	7

3 Geometry

3.1 Primitives

Module/Class	Function/Method	Page
em	Anchor	10
em	Axis	8
em	CoordinateSystem	9
em	Line	10
em	Plane	9
em.Axis	construct_cs	9
em.Axis	cross	9
em.Axis	dot	9
em.Axis	pair	9
em.CoordinateSystem	affine_from_global	10
em.CoordinateSystem	affine_to_global	10
em.CoordinateSystem	displace	9
em.CoordinateSystem	flip_z	9
em.CoordinateSystem	in_global_basis	10
em.CoordinateSystem	in_global_cs	10
em.CoordinateSystem	in_global_cs	10
em.CoordinateSystem	in_local_basis	10
em.CoordinateSystem	rotate	9
em.CoordinateSystem	swapxy	9
em.Line	cmid	10
em.Line	cpoint	10
em.Line	from_path	10
em.Line	from_points	10
em.Line	line_integral	11
em.Line	smooth	10
em.Line	subsamample	10
em.Plane	cs	9
em.Plane	flip	9
em.Plane	grid	9
em.Plane	span	9

3.2 3D Geometry

Module/Class	Function/Method	Page
em.geo	Box	11
em.geo	CoaxCylinder	11
em.geo	Cone	11
em.geo	Cylinder	11
em.geo	HalfSphere	11
em.geo	Horn	11
em.geo	Sphere	11
em.geo	StepItems	11
em.geo	XYPolygon	12
em.geo	open_pml_region	11
em.geo	open_region	11
em.geo	pmlbox	12

3.3 2D Geometry

Module/Class	Function/Method	Page
em.geo	Disc	12
em.geo	Plate	12
em.geo	XYPlate	12

3.4 Operations

Module/Class	Function/Method	Page
em.geo	add	13
em.geo	add	13
em.geo	bounding_box	14
em.geo	change_coordinate_system	14
em.geo	embed	13
em.geo	extrude	14
em.geo	intersect	13
em.geo	mirror	14
em.geo	remove/subtract	13
em.geo	rotate	13
em.geo	stick	14
em.geo	stretch	14
em.geo	translate	13
em.geo	unite	14

3.5 XYPolygon

Module/Class	Function/Method	Page
XYPolygon	Revolve	12
XYPolygon	circle	13
XYPolygon	connect	13
XYPolygon	extend	12
XYPolygon	extrude	12
XYPolygon	fillet	12
XYPolygon	geo	12
XYPolygon	incs	12
XYPolygon	iterate	12
XYPolygon	parametric	13
XYPolygon	rectangle	13
XYPolygon	shrink	13

3.6 PCB Layouting

Module/Class	Function/Method	Page
em.geo	PCBLayer	14
em.geo	PCBNew	14
em.geo.PCBNew	add_hole	16
em.geo.PCBNew	add_poly	16
em.geo.PCBNew	add_vias	14
em.geo.PCBNew	compile_paths	16
em.geo.PCBNew	determine_bounds	15
em.geo.PCBNew	generate_air	15
em.geo.PCBNew	generate_pcb	15
em.geo.PCBNew	generate_vias	15
em.geo.PCBNew	load	14
em.geo.PCBNew	lumped_port	15
em.geo.PCBNew	lumped_port_pts	15
em.geo.PCBNew	modal_port	15
em.geo.PCBNew	new	15
em.geo.PCBNew	plane	15
em.geo.PCBNew	radial_stub	15
em.geo.PCBNew	set_bounds	15
em.geo.PCBNew	z	14

3.7 StripPath

In practice you will call these methods after calling the `pcb.new(...)` function. Not on a manually created `StripPath` object. That won't work.

Module/Class	Function/Method	Page
em.geo.StripPath	bck	16
em.geo.StripPath	curve	16
em.geo.StripPath	end	16
em.geo.StripPath	jump	17
em.geo.StripPath	lumped_element	17
em.geo.StripPath	merge	17
em.geo.StripPath	pturn	16
em.geo.StripPath	short	17
em.geo.StripPath	skip	16
em.geo.StripPath	split	16
em.geo.StripPath	start	16
em.geo.StripPath	store	16
em.geo.StripPath	straight	16
em.geo.StripPath	stub	17
em.geo.StripPath	taper	16
em.geo.StripPath	to	17
em.geo.StripPath	turn	16
em.geo.StripPath	via	17
em.geo.StripPath	xs/ys	16

4 Meshing

Module/Class	Function/Method	Page
em.Mesher	set_boundary_size	18
em.Mesher	set_curved_boundary_meshing	17
em.Mesher	set_domain_size	18
em.Mesher	set_face_size	18
em.Mesher	set_pec_face	18
em.Mesher	set_size	18
em.Mesher	set_trace_refinement	18
em.Mesher	unset_constraints	17
em.Simulation	set_resolution	17

5 Physics

5.1 Microwave Physics

Module/Class	Function/Method	Page
Microwave3D	eigenmode	19
Microwave3D	modal_analysis	18
Microwave3D	reset	18
Microwave3D	run_adaptive_sweep	19
Microwave3D	run_scattered	19
Microwave3D	run_sweep	19
Microwave3D	set_frequency	18
Microwave3D	set_frequency_range	18
Microwave3D	set_resolution	18

5.2 Boundary Conditions

The following functions are called on:

python

```
1 pec_bc = my_model.mw.bc.PEC(...) # etc
```

Module/Class	Function/Method	Page
MWBoundaryConditionSet	AbsorbingBoundary	20
MWBoundaryConditionSet	CoaxPort	22
MWBoundaryConditionSet	FloquetPort	21
MWBoundaryConditionSet	LumpedElement	20
MWBoundaryConditionSet	LumpedPort	22
MWBoundaryConditionSet	ModalPort	21
MWBoundaryConditionSet	PEC	19
MWBoundaryConditionSet	PMC	19
MWBoundaryConditionSet	RectangularWaveguide	21
MWBoundaryConditionSet	ScatteredField	20
MWBoundaryConditionSet	SurfaceImpedance	20
MWBoundaryConditionSet	ThinConductor	20
MWBoundaryConditionSet	UserDefinedPort	22
MWBoundaryConditionSet	floquet_port	21
ModalPort	align_modes	21
ModalPort	set_integration_line	21
ModalPort	set_terminals	21
RectangularWaveguide	align_modes	21
ScatteredField	set_excitations	20

6 Data

6.1 Generic

Module/Class	Function/Method	Page
em	MWData	22
em	MWField	22
em.MWData	export_farfields	22
em.MWData	merge_with	22
em.MWField	boundary	23
em.MWField	combine_ports	23
em.MWField	current_boundary	23
em.MWField	cutplane	23
em.MWField	cutplane_normal	24
em.MWField	excite_port	22
em.MWField	farfield	24
em.MWField	farfield_2d	24
em.MWField	farfield_3d	24
em.MWField	grid	24
em.MWField	int_line	23
em.MWField	int_lumped_element	23
em.MWField	int_surf	23
em.MWField	int_vol	23
em.MWField	interpolate	23
em.MWField	relative	22
em.MWField	set_excitations	23

6.2 EHField and EHFieldFF classes

The **EHField** class contains all the different possible electric and magnetic fields (E,H,D,B,P,S,Sm,J,Q) and the coordinates where they where computed. It also contains metadata on the structure of the underlying coordinates (2D grid, 3D grid, unstructured grid, triangulated surface etc.).

All field attributes are: E, H, S, P, D, B, Ex, Ey, Ez, Hx, ... etc, Sm (time averaged Poynting vector), Js, Jv, normE, normB, normS, normSm etc.

The **EHFieldFF** class contains the following farfield properties: E, H, Ex, Ey, Ez, Hx, Hy, Hz, Etheta, Ephi, Htheta, Hphi, Erhcp, Elhcp, AR, gain, dir, RCS.

Some attributes allow for further sub attribute specifications as they return an object of type `FarFieldComponent`. This class itself has the attributes: x, y, z, theta, phi, lhcp, rhcp and norm. For example:

python

```
1 farfield = field.farfield_3d()
2 farfield.Etheta #one way
3 farfield.E.theta #using FarFieldComponent
4 farfield.gain.theta
5 farfield.dir.lhcp
```

Technicall the EHfield and EHFieldFF classes are not part of EMerge but part of the shared `emsutil` library which is also used by Optycal.

Module/Class	Function/Method	Page
em	EHField	24
em	EHFieldFF	25
em	FarFieldComponent	25
em.EHField	scalar	25
em.EHField	vector	24
em.EHFieldFF	E	25
em.EHFieldFF	H	25
em.EHFieldFF	ang	25
em.EHFieldFF	dir	25
em.EHFieldFF	gain	25
em.EHFieldFF	surfplot	25
em.EHFieldFF	total_radiated_power_integral	25

7 Display

Module/Class	Function/Method	Page
EMergeDisplay	add_clip_volume	28
EMergeDisplay	add_cloud	28
EMergeDisplay	add_contour	28
EMergeDisplay	add_quiver	28
EMergeDisplay	add_scatter	27
EMergeDisplay	add_streamline	28
EMergeDisplay	add_surf	27
EMergeDisplay	add_text	29
EMergeDisplay	add_title	28
EMergeDisplay	add_trisurf	27
EMergeDisplay	animate	27
EMergeDisplay	cbar	26
EMergeDisplay	drawing_bw	27
EMergeDisplay	save_vtk	27
EMergeDisplay	set_theme	26
EMergeDisplay	show	26
PVDisplay	add_anchors	25
PVDisplay	add_farfield3d	26
PVDisplay	add_field	27
PVDisplay	add_line	26
PVDisplay	add_object	25
PVDisplay	add_objects	26
PVDisplay	add_portmode	26
PVDisplay	add_scatter	26
PVDisplay	populate	26

8 Global

8.1 Global States and Definitions

function em.cleanup()

Clean-up all variables from the global state thus reducing the RAM requirements to the minimum possible.

Parameters

Returns: None

Raises: -

property em.GCS

The Global coordinate system object. See `CoordinateSystem`.

Returns: em.CoordinateSystem

property em.CORNER

An Enum object that is used by CAD modelling to tell EMerge which point of a rectangle or box to align the position with. The corner corresponds to the most -X, -Y and -Z point (lower, left, front).

Example:

python

```
1 box = em.geo.Box(1,1,1,(0, 2, 5), alignment=em.CORNER)
```

Returns: em.Alignment

property em.CENTER

An Enum object that is used by CAD modelling to tell EMerge which point of a rectangle or box to align the position with.

Example:

python

```
1 box = em.geo.Box(1,1,1,(0, 2, 5), alignment=em.CENTER)
```

Returns: em.Alignment

property em.EMSolver

An Enum class containing all the different solvers. Available values are:

- `.SUPERLU` - Default Sparse LU in Scipy
- `.UMFPACK` - Requires `scikit-umfpack`
- `.PARDISO` - Default parallel Sparse LU for x86
- `.CUDSS` - NVidias GPU based Sparse LU solver. Requires Cupy dependencies through `pip install emerge[cudss]`
- `.MUMPS` - MUMPS solver for MacOS only. Use `AASDS` as its faster and easier to install.
- `.AASDS` - Interface for the Apple Accelerate sparse LU solver. (install using `pip install git+https://github.com/FennisRobert/emerge-aasds`)
- `.BICGSTAB` - Scipy's Bi-conjugate Gradient Stabilized iterative solver. Heatconduction only.
- `.CG` - Scipy's Conjugate Gradient iterative solver. Heatconduction only.
- `.CHOLMOD` - Cholesky solver. Requires `scikit-sparse`
- `.LAPACK` (eigenvector solver)
- `.ARPACK` (eigenvector solver)
- `.SMART_ARPACK` (eigenvector solver)
- `.SMART_ARPACK_BMA` (eigenvector solver)

Returns: em.EMSolver

8.2 Simulation Class

function em.Simulation()

Creates an EMerge Simulation object. Used as the central point of contact for all simulation related activities.

Parameters

- modelname: str - The name of the simulation model. Used internally to separate different Simulation objects and used to save and load the correct data.
- loglevel: "TRACE" | "DEBUG" | "INFO" | "WARNING" | "ERROR" - The loglevel of the logger. Defaults to "INFO". All messages of that type importance and greater will be shown in the terminal.
- load_file: bool = False - If the simulation should immediately load the data from the file assuming it exists.
- save_file: bool = False - Save the file automatically if the Simulation object exits a "with" context manager.
- write_log: bool = False - If a log file should be automatically created. Log file level is TRACE.
- path_suffix: str = ".EMResult" - The path suffix used when storing or loading data.
- store_system: "msgpack" | "joblib" - Which file storing system to use. "joblib" is most stable but can store unsafe data. Importing files from external parties generated with "joblib" may contain dangerous executable code. "msgpack" is a custom binary file system mapping all EMerge data to Python primitives such as dicts, lists and numpy arrays. May cause problems with some classes if they still contain Callable objects that cannot be serialized. This is always safe.

Returns: Simulation

Raises: -

function em.Simulation.ping()

Send a message to the NTFY app system (downloadable on Android and iPhone). These are public messages that will be send to people who subscribe to channels. Use a custom message and channel name to notify if a simulation is finished for example.

IMPORTANT: As said, these channels are public. Anyone can subscribe to channels you send messages to. Do not share any sensitive information in these channels. Use very hard to predict channel names to keep things as private as possible.

Parameters

- message: str - The message
- channel: str - The channel

Returns: None

Raises: -

function em.Simulation.reset()

Whipe internal simulation data.

Parameters

- mesh: bool = False - If the mesh should be deleted
- geometry: bool = False - If the Geometry should be deleted and GMSH should be reset.
- physics: bool = False - If the physics and boundary conditions should be deleted.
- data: bool = False - If all simulation data should be deleted
- all: bool = False - If everything should be deleted.

Returns: None

Raises: -

function em.Simulation.cache_build()

Checks if all the lines in the .py file above this line are the same. If they are this function returns false. Can be used to cache the entire build and simulation process of a simulations:

python

```
1 if model.cache_build():
2     box = em.geo.Box(...)
3     model.commit_geometry()
4     # etc
5     # all simulation steps here
6     data = model.mw.run_sweep()
7     model.save()
8 else:
9     data = model.data.mw
```

Parameters • message: str - The message
• channel: str - The channel

Returns: bool

Raises: -

function em.Simulation.cache_run()

Checks if all the lines in the .py file above this line are the same. If they are this function returns false. Can be used to cache the simulation solve steps:

python

```
1 if model.cache_run():
2     data = model.mw.run_sweep()
3     model.save()
4 else:
5     data = model.data.mw
```

Parameters • message: str - The message
• channel: str - The channel

Returns: bool

Raises: -

function em.Simulation.check_version()

Crashes the python script if the version provided in this argument is more recent than the EMerge version. Used in examples to alert people that the example files they are using are incompatible with the Emerge version for example.

Parameters • target_version: str - For example “2.7.1”

Returns: None

Raises: -

function em.Simulation.set_physics()

Tells EMerge which physics are used. Mostly important to deactivate Microwave physics and activate Heat Conduction for example when EMerge defines the default mesh size. Microwave is turned on by default. In these cases a heat-conduction simulation only would require a simulation frequency because EMerge thinks it is going to do a microwave simulation.

Parameters • microwave: bool - If you are running microwave simulations
• heatconduction: bool - If you are running heatconduction simulations.

Returns: None

Raises: -

function em.Simulation.activate()

Tells EMerge which solution number of parameter to activate as the current geometry/mesh. In case of parameter sweeps you can use this to pick a specific parameter permutation.

Parameters • _indx: int | None = None - A specific solution index
• variables - A list of keyword variables. For example: `width=10*mm` for some parameter sweep iteration.

Returns: None

Raises: -

function em.Simulation.save()

Saves the current Simulation object to a file.

Parameters • _force_save: bool = True - Will always save the file no matter what. If set to False it will only save a file if actually new simulation results are generated.
• screenshot: bool = False - Will cause a screenshot of the simulation model to also be saved. Has some annoying consequences to hijacking PyVista later in the script.

Returns: None

Raises: -

function em.Simulation.load()

Orders EMerge to load all data.

Parameters

Returns: None

Raises: -

function em.Simulation.set_loglevel()

Set a new log level.

Parameters • loglevel = “TRACE” | “DEBUG” | “INFO” | “WARNING” | “ERROR”

Returns: None

Raises: -

function em.Simulation.view()

Shows the 3D geometry as it is currently. By Default it tries to use EEmerges PyVista based viewer. If there is no Mesh yet it will generate a very coarse one. If you have a geometry state that cannot be meshed, set `use_gmsh=True` to use GMSH’s build in GUI which can display the Geometry without requiring a mesh step.

Parameters • selections: list[Selection] | None - A list of selections to highlight-color
• use_gmsh: bool = False - If the GMSH geometry viewer should be used.
• plot_mesh: bool = False - If the mesh should be shown instead of the geometry volumes.
• volume_mesh: bool = True - If true then plot_mesh will plot the volume mesh, if False it will only plot the mesh on the surface of objects.
• opacity: float | None = None - An upper limit to the opacity of geometries
• labels: bool = False - If true, labels will be added to each geometry showing the geometry names.
• bc: bool = False - If True, EMerge will draw textures on surface with boundary condition name labels
• bw: bool = False - If True, the view will be black and white and with an optimized style for printing on black and white paper.
• face_labels: bool = False - If true EMerge will add labels to all the different face names in the geometry
• screenshot: str | None = None - If a filename is provided, EMerge will automatically generate a screenshot to a file with the provided filename
• assigned_materials: bool = False - If False (default) EMerge will draw all geometries as defined, overlapping with their respective materials. If True, Emerge will draw all disjoint volume elements with the actual material rendered as will be simulated. In case of overlapping geometries, this can be used to validate if the correct materials are assigned.
• off_screen: bool = False - If true, no display will be shown. Used in conjunction with .screenshot().

Returns: None

Raises: -

function em.Simulation.set_periodic_cell()

Provide a PeriodicCell object such as `HexCell` and `RectCell` to automatically configure simulations with periodic boundaries.

Parameters • cell: PeriodicCell - The periodic cell object.

Returns: None

Raises: -

function em.Simulation.**commit_geometry()**

Tell EMerge that you are done defining the geometries. This will initiate a boolean fragment and **has** to be called before generating a mesh or defining seletions or boundary conditions.

Parameters
Returns: None
Raises: -

function em.Simulation.**all_geos()**

Returns a list of all geometries

Parameters
Returns: list[GeoObject]
Raises: -

function em.Simulation.**all_boundaries()**

Returns a selection object of all boundaries

Parameters
Returns: FaceSelection
Raises: -

function em.Simulation.**find_geo()**

Search for a geometry by name assigned. Searches for an exact match first and then a partial match defined as any geometry name that contains the provided name. First result is returned.

Parameters	name: str
Returns:	GeoObject
Raises:	-

function em.Simulation.**quick_mesh()**

Generates a quick mesh that does not require simulation frequency details.

Parameters
Returns: None
Raises: -

function em.Simulation.**generate_mesh()**

Tells EMerge to start generating the 3D mesh. Will not regenerate by default if a (non quick_mesh) mesh already exists. Also assigns the default boundary conditions when done.

Parameters	regenerate: bool = False - If the mesh should be regenerated.
Returns:	None
Raises:	-

function em.Simulation.**parameter_sweep()**

Used as an iterator to auotmate parameter sweep simulations. Provide a list of keyword arguments with iterables that are to be swepted over. With multiple variables, EMerge will **always** take the outer product (permutation of all variable combinations). This iterator will yield a tuple of each variable that can be used in the simulation. **The variables are returned in alphabetical order** because Python dicts do not preserve order by default.

Example usage:

python

```
1 sim = em.Simulation(...)
2 for width in sim.parameter_sweep(True, width=[1*mm, 2*mm, 3*mm]):
3     box = em.geo.Box(width, 1, 2)
4     # etc
5     sim.mw.run_sweep()
6 data = sim.data.mw
7 field = data.field.find(freq=1e9, width=2*mm)
```

- | | |
|-------------------|---|
| Parameters | <ul style="list-style-type: none">clear_mesh: bool = True - If the mesh and geomometry should be reset in each iteration. Can be set to False in simulations where you might only change boundary condition settings, not the geometry itself.**parameters - A sequence of optional keyword arguments that are the variable name and the set of variables. |
|-------------------|---|

Returns: None

Raises: -

function em.Simulation.**export()**

Export the current Geometry or Mesh. Uses GMSHs built in save system. Supported extensions are:

3D Model:

.opt, .geo_unrolled, .brep, .xao ,.step and .iges

Mesh:

.msh, .inp, .key, .celum, .cgns, .diff, .unv, .ir3, .mes, .mesh, .mail, .m, .bdf, .off, .p3d, .stl, .wrl, .vtk, .dat, .ply2, .su2, .neu, .x3d

- | | |
|-------------------|--|
| Parameters | <ul style="list-style-type: none">filename: str - The filename plus extension for the file type. |
|-------------------|--|

Returns: None

Raises: -

function em.Simulation.**set_solver()**

Provide a solver object or solver enum to use as preferres sparse solver.

Exapmle:

python

```
1 sim.set_solver(em.EMSolver.PARDISO)
```

- | | |
|-------------------|--|
| Parameters | <ul style="list-style-type: none">solver: EMSolver Solver - The solver to use. See <code>EMSolver</code> |
|-------------------|--|

Returns: None

Raises: -

function em.Simulation.**from_simulations()**

Construct a new simulation object but import the simulation date from multiple different other simulation objects.

- | | |
|-------------------|--|
| Parameters | <ul style="list-style-type: none">name: str - The Name of the new simulation object*sims: Simulation - A sequence of other simulation objects to merge.**kwargs - All other standard keyword arguments when constructing a <code>Simulation</code> object. |
|-------------------|--|

Returns: None

Raises: -

9 Geometry Functions

function em.**Axis()**

Creates an Axis object which is a normalized 3D vector with length 1. Globally available realizations are: em.XAX, em.YAX and em.ZAX

- | | |
|-------------------|--|
| Parameters | <ul style="list-style-type: none">vector: np.ndarray Iterable[float, float, float] |
| Returns: | Axis |
| Raises: | - |

function em.Axis.**cross**()

Take the cross product with another axis.

Parameters • other: Axis
Returns: Axis
Raises: -

function em.Axis.**dot**()

Take the dot product with another axis.

Parameters • other: Axis
Returns: Axis
Raises: -

function em.Axis.**pair**()

Pair with another Axis object to create a Plane object

Parameters • other: Axis
Returns: Plane
Raises: -

function em.Axis.**construct_cs**()

Creates a CoordinateSystem object at origin (xo,yo,zo) such that the Z-axis is parallel to the Axis.

Parameters • xo: float = 0.0
• yo: float = 0.0
• zo: float = 0.0
Returns: CoordinateSystem
Raises: -

function em.Plane()

Creates a 2D plane spanned by two Axis objects which need not be orthogonal. All permutations of XYPLANE, YZPLANE are defined in the global space (em.XYPLANE etc.).

Parameters • uax: Axis
• vax: Axis
Returns: Plane
Raises: -

function em.Plane.**flip**()

Flips the plane axes

Parameters
Returns: Axis
Raises: -

function em.Plane.**cs**()

Create a CoordinateSystem object at origin (xo, yo, zo) such that the two axes span the XY plane

Parameters • xo: float = 0.0
• yo: float = 0.0
• zo: float = 0.0
Returns: CoordinateSystem
Raises: -

function em.Plane.**grid**()

Creates the (N,M) shaped rectangular grid point set based on arguments that match `np.linspace`.

Parameters • uax: tuple[float, float, int] - The the start - end values and the number of points along the u-axis
• vax: tuple[float, float, int] - the start to end values and the number of points along the v axis.
• origin: tuple[float, float, float] - The origin in 3D coordinates

indexing: “xy”, “ij”
Returns: tuple[np.ndarray, np.ndarray, np.ndarray] - X,Y,Z grid arrays
Raises: -

function em.Plane.**span**()

Similar to `grid` but uses an equal number in of points along the u/v axes and starts in 0,0.

Parameters • u: float - The grid u-value range
• v: float - the grid v-value range
• N: int - The number of points along each axis
• origin: tuple[float, float, float] - the origin for the start of the grid.
Returns: tuple[np.ndarray, np.ndarray, np.ndarray] - X,Y,Z grid arrays
Raises: -

function em.**CoordinateSystem**()

Creates a CoordinateSystem object from a set of axis objects and an origin.

Parameters • xax: em.Axis
• yax: em.Axis
• zax: em.Axis
• origin: np.ndarray
Returns: CoordinateSystem
Raises: -

function em.CoordinateSystem.**flip_z**()

Inverts the Z-axis

Parameters
Returns: CoordinateSystem
Raises: -

function em.CoordinateSystem.**displace**()

Displace the coordinate system origin

Parameters • dx: float = 0.0
• dy: float = 0.0
• dz: float = 0.0
Returns: CoordinateSystem
Raises: -

function em.CoordinateSystem.**rotate**()

Rotate the coordinate system around an axis

Parameters • axis: tuple | em.Axis - The rotation axis
• angle: float - The rotation angle
• degrees: bool = True - If the angle is in degrees or not.
• origin: bool = False - If the origin itself should be rotated or just the basis vectors.
Returns: CoordinateSystem
Raises: -

function em.CoordinateSystem.**swapxy**()

Swaps the X and Y axes

Parameters
Returns: CoordinateSystem
Raises: -

function em.CoordinateSystem.**affine_from_global()**

Returns an affine transformation matrix which would transform coordinate 4-vectors from the global coordinate system into this one.

Parameters
Returns: np.ndarray
Raises: -

function em.CoordinateSystem.**affine_to_global()**

Returns an affine transformation matrix which would transform coordinate 4-vectors from this coordinate system into the global coordinate system.

Parameters
Returns: np.ndarray
Raises: -

function em.CoordinateSystem.**in_global_cs()**

Convert arrays of x,y and z coordinate values into the global coordinate system.

Parameters • x: float | np.ndarray
• y: float | np.ndarray
• z: float | np.ndarray
Returns: tuple[np.ndarray, np.ndarray, n]
Raises: -

function em.CoordinateSystem.**in_global_cs()**

Convert arrays of x,y and z coordinate values from the global coordinate system.

Parameters • x: float | np.ndarray
• y: float | np.ndarray
• z: float | np.ndarray
Returns: tuple[np.ndarray, np.ndarray, n]
Raises: -

function em.CoordinateSystem.**in_global_basis()**

Convert arrays of x,y and z vector components into the global coordinate system basis (does not use the origin).

Parameters • x: float | np.ndarray
• y: float | np.ndarray
• z: float | np.ndarray
Returns: tuple[np.ndarray, np.ndarray, n]
Raises: -

function em.CoordinateSystem.**in_local_basis()**

Convert arrays of x,y and z vector components from the global coordinate system (does not use the origin).

Parameters • x: float | np.ndarray
• y: float | np.ndarray
• z: float | np.ndarray
Returns: tuple[np.ndarray, np.ndarray, n]
Raises: -

function em.**Anchor()**

Creates and Anchor object. An anchor is a generalization of a coordinate system but embued with properties. They created for geometries on important positions. Anchors can be used to align certain parts of objects with other parts..

Parameters • co: np.ndarray - The origin of the Anchor point
• x: np.ndarray - The x-axis of the anchor point
• y: np.ndarray - The y-axis of the anchor point
• z: np.ndarray - THe z-axis of the anchor point.
Returns: tuple[np.ndarray, np.ndarray, n]
Raises: -

function em.**Line()**

Creates a line object that can be used to perform path integrals. The Line is always represented as a segmented list of straight line segments.

Parameters • xpts: np.ndarray - A list of x-coordinates.
• ypts: np.ndarray - A list of y-coordinates.
• zpts: np.ndarray - A list of z-coordinates.
Returns: Line
Raises: -

property em.Line.**cmid**

The midpoints of all line segments.

Returns: tuple[np.ndarray, np.ndarray, np.ndarray]

property em.Line.**cpoint**

The coordinates of the line vertices.

Returns: tuple[np.ndarray, np.ndarray, np.ndarray]

function em.Line.**from_points()**

A static method construction to create a straight line from a start to an end point.

As line integrals are performed as a discretized integral. Make sure to pick enough points to capture all the detail.

Parameters • start: np.ndarray - A (3,) numpy array for the start point
• end: np.ndarray - A (3,) numpy array for the end point
• Npts: int - The total number of points with which to segment the line.
Returns: Line
Raises: -

function em.Line.**from_path()**

An alternative constructor for a line that connects a set of coordinates and adds points on each segment to respect a maximum line segment size.

Parameters • *points: tuple[float, float, float] - A sequence of 3D coordinates for each path point.
• ds: float - The maximum segment size for a path.
Returns: Line
Raises: -

function em.Line.**subsammple()**

Splits the line segments in n equal parts.

Parameters • n: int - The number of subsegments
Returns: Line
Raises: -

function em.Line.**smooth()**

Perform a laplacian smoothing to the line. While preserving the arc length.

Each step replaces every interior point with the average of its two neighbors then rescales all segment lengths uniformly so the total arc length is unchanged. For closed loops(where the first and last point coincide), all points are treated as interior.

This function is created by Claude Code.

Parameters • nsteps: int = 1 - The number of smoothing steps.
Returns: Line
Raises: -

function em.Line.**line_integral()**

Performs a line integral of some arbitrary vector field or scalar field. The function must respect the signatures:

- fn(x,y,z) → (3,N)
- fn(x,y,z) → (N,)

Parameters • evalfunc: Callable - The function to integrate
Returns: float | complex
Raises: -

function em.geo.**Box()**

Creates a 3D box object.

Parameters • width: float
• depth: float
• height: float
• position: tuple(float, float, float)
• alignment: em.CENTER | em.CORNER
Returns: GeoVolume
Raises: -

function em.geo.**Sphere()**

Creates a 3D sphere object with the center of the sphere at the provided position.

Parameters • radius: float
• position: tuple[float, float, float],
Returns: GeoVolume
Raises: -

function em.geo.**HalfSphere()**

Creates a hemispherical geometry with the center of the sphere at the provided position. The Sphere is cut in the plane defined by the direction argument.

Parameters • radius: float
• position: tuple[float, float, float],
• direction: tuple | Axis, the normal axis of the cutplane of the hemisphere
Returns: GeoVolume
Raises: -

function em.geo.**Cylinder()**

Creates a 3D Cylinder. The cylinder starts in the (o,o,o) of the local coordinate system and is aligned with the Z-axis.

Parameters • radius: float
• height: float
• cs: em.CoordinateSystem,
• Nsections: int | None, the number of polygonal sections instead of a curved perimeter.
Returns: GeoVolume
Raises: -

function em.geo.**CoaxCylinder()**

Creates a Coaxial 3D Cylinder with the core cutout of the geometry. The cylinder starts in the (o,o,o) of the local coordinate system and is aligned with the Z-axis. Pairs well with [em.coax_rin\(\)](#) and [em.coax_rout\(\)](#) to compute the radii.

Parameters • rout: float
• rin: float
• height: float
• cs: em.CoordinateSystem,
• Nsections: int | None, the number of polygonal sections instead of a curved perimeter.
Returns: GeoVolume
Raises: -

function em.geo.**Cone()**

Creates a 3D cone

Parameters • po: tuple[float, float, float] - the start coordinate
• direction: tuple[float, float, float] - the direction of the cone.
• r1: float - The bottom radius
• r2: float - the tip radius
Returns: GeoVolume
Raises: -

function em.geo.**Horn()**

Creates a pyrimidal horn such that the horn aperture is in the local XY plane with the horn extruding in the positive Z-direction.

Parameters • start_aperture: tuple[float, float] - the starting aperture width, height
• end_aperture: tuple[float, float] - The ending aperture width, height
• height: the height/length of the horn
• cs: em.CoordinateSystem - The local coordinate system of the horn.
Returns: GeoVolume
Raises: -

function em.geo.**open_region()**

Convenience function to create an airbox around the current geometry.

Parameters • xmargin: float | tuple[float, float] - either the margin on both sides (X-axis) or -X and +X respectively
• ymargin: float | tuple[float, float]
• zmargin: float | tuple[float, float]
• material: em.Material (default to AIR)
Returns: GeoVolume
Raises: -

function em.geo.**open_pml_region()**

Convenience function to create an airbox around the current geometry plus PML regions. See [pmlbox](#)

Parameters • xmargin: float | tuple[float, float] - either the margin on both sides (X-axis) or -X and +X respectively
• ymargin: float | tuple[float, float]
• zmargin: float | tuple[float, float]
• material: em.Material (default to AIR)
Returns: GeoVolume
Raises: -

function em.geo.**StepItems()**

Open a .step file. Geometries will be accesible through the StepItems interface.

Parameters • name: str - Name used in EMerge for the geometries
• filename: str - The filename of the .step file
• unit: float = 1.0 - The unit in which the step file is expressed. Sometimes this can be mm (0.001).
• string_parser: Callable | None - An optional function to sanitize the strings of object names. Must be a function that takes a string and returns a new string.
Returns: StepItems
Raises: -

function em.geo.pmlbox()

Creates an air box plus the desired PML region boxes. The sides that should get a PML layer can be defined either by the sides argument or the top/bottom/.../right arguments.

Parameters

- width: float
- depth: float
- height: float
- position: tuple[float, float, float]
- alignemnt: em.CENTER | em.CORNER
- material: em.MATERIAL (default AIR)
- thickness: float - the thickness of the PML layers
- Nlayers: int (default 1) - The number of PML box layers. Creates complex geometries but can improve performance.
- N_mesh_layers: Computes the local mesh size to realize N layers in the extrusion direction. This also influences the size in all directions which can explode the mesh. Increase the thickness to prevent a mesh explosion.
- exponent: float = 1.5 - The PML material scaling exponent
- deltamax: float = 8.0 - A reflection tuning parameters
- sides: str = “. An list of which sides to add a PML layer to using the convention ([T]op, [B]ottom, [L]eft, [R]ight, [F]ront, b[A]ck)
- top, bottom, left, right, front back: bool = False - which face to add.
- .

Returns: list[GeoVolume]

Raises: -

function em.geo.XYPolygon()

Define a polygon in a non-embedded separate XY coordinate system. Can later be embedded in 3D space by using the .geo or .extrude arguments. See Section 3.5.

Parameters

- xs: Iterable[float],
- ys: Iterable[float],
- cs: em.CoordinateSystem - Can optionally provide the coordinate system here already.
- resolution: float - The discretization resolution for curved boundaries.

Returns: XYPolygon

Raises: -

function em.geo.XYPlate()

Creates a rectangular plate aligned along the X and Y axis of a local coordinate system.

Parameters

- width: float
- height: float
- position: tuple[float, float, float]
- alignment: em.CENTER | em.CORNER

Returns: GeoSurface

Raises: -

function em.geo.Plate()

Creates a rectangular plate in an arbitrary 3D orientation using a starting origin and then two protruding u anv v axes.

Parameters

- origin: tuple[float, float, foat]
- radius: float
- axis: tuple[float, float, float] the disc normal axis
- radius_opt: float | None - The minor ellipse axis (radius becomes major)
- axis_opt: tuple[float, float, float] - Specifies the direction of the major axis.

Returns: GeoSurface

Raises: -

function em.geo.Disc()

Creates a 2D round or elliptical disc.

Parameters

- origin: tuple[float, float, foat]
- u: tuple[float, float, float]
- v: tuple[float, float, float]

Returns: GeoSurface

Raises: -

function XYPolygon.incs()

Provide a coordinate system for the XYPolygon object then return it.

Parameters

- cs: em.CoordinateSystem

Returns: XYPolygon

Raises: -

function XYPolygon.extend()

Extend the list of x and y coordinates with a new list of xy coordinates and return the object.

Parameters

- xpts: list[float]
- ypts: list[float]

Returns: XYPolygon

Raises: -

function XYPolygon.iterate()

Iterate over all the XY points as a tuple of coordinates.

Parameters

- Returns: Generator[tuple[float, float]]

Raises: -

function XYPolygon.fillet()

Create a fillet (rounded corner) with a given radius for the points at the provided indices. Example:

xypolygon.fillet(1*mm, 0,1,5,6,10)

Parameters

- radius: float
- indices: int, a sequence of indices.

Returns: XYPolygon

Raises: -

function XYPolygon.extrude()

Extrude the XYPolygon a certain distance in the local coordinate system positive Z-direction.

Parameters

- length: float - the Extrusion distance
- cs: em.CoordinateSystem | None - An optional specification of the coordinate system. Defaults to the previously defined or otherwise the global coordinate system.

Returns: GeoPrism

Raises: -

function XYPolygon.geo()

Embed the current polygon in 3D space by creating a polygon GeoSurface

Parameters

- cs: CoordinateSystem - An optional specification of the coordinate system if it wasn't already provided. Defaults to the global coordinate system GCS

Returns: GeoSurface

Raises: -

function XYPolygon.Revolve()

First embed the XYPolygon as a surface in 3D space and then revolve it around a revolution axis.

Parameters

- cs: CoordinateSystem - An optional specification of the coordinate system if it wasn't already provided. Defaults to the global coordinate system GCS
- origin: tuple[float, float, float] - The origin of the revolution axis in global coordinates.
- axis: tuple[float, float, float] - The revolution axis
- angle: float = 360 - The revolution angle in degrees. Defaults to 360

Returns: GeoObject

Raises: -

function XYPolygon.**circle**()

Static method to automatically generate an XYPolygon object for a circle approximated by N sections. The number of sections can be controlled in three ways, either by providing a section edge size, tolerance form the curve or the number of sections.

Parameters

- radius: float
- dsmax: the maximum section edge length - optional
- tolerance: the maximum error tolerance for the N-sectional polygon to deviate from the ideal curve.
- Nsections: The number of sections.

Returns: XYPolygon

Raises: -

function XYPolygon.**rectangle**()

Static method to create a simple rectangle

Parameters

- width: float
- height: float
- origin: tuple[float, float, float]
- alignment: em.CENTER | em.CORNER

Returns: XYPolygon

Raises: -

function XYPolygon.**parametric**()

Appends the polygon with a new curve section defined by a parametric function set

$$x(t), y(t) \forall t \in [tmin, tmax]$$

The detail of the approximation is controlled by xmin and tolerance.

Parameters

- xfunc: Callable - A parametric function that returns an x-coordinate as a function of float t.
- yfunc: Callable - Same as for xfunc
- xmin: float = 0.001 - The minimum segment size
- tolerance: float = 1e-5 - The amount the segmented line section can deviate from the ideal curve.
- tmin: float = 0.0 - The start value for the parametric line.
- tmax: float = 1.0 - The end value for the parametric line.

reverse: bool = False - If the curve should be traversed in reverse (tmax to tmin).

Returns: XYPolygon

Raises: -

function XYPolygon.**connect**()

Connect two XYPolygons with a defined coordinate system creating a pipe between them

Parameters

- other: XYPolygon - The XYPolygon object to connect this object to

Returns: GeoVolume

Raises: -

function XYPolygon.**shrink**()

Shrinks the XY polygon by making each line segment move some distance inwards. This function does not safely take into account the possibility of segments disappearing.

Parameters

- distance: float - The distance to move the polygon inwards.

Returns: XYPolygon

Raises: -

function em.geo.**add**()

Perform a boolean union between two GeoObjects. Must be of the same dimension.

Parameters

- main: GeoObject
- other: GeoObject
- remove_object: bool = True - If the main object should be removed after union.
- remove_tool: bool = True - If the tool object should be removed after union.

Returns: GeoObject

Raises: -

function em.geo.**remove/subtract**()

Function goes by both names `remove` and `subtract`.

Perform a boolean difference between two GeoObjects. Subtracts the tool from the main object. Must be of the same dimension.

Parameters

- main: GeoObject
- other: GeoObject
- remove_object: bool = True - If the main object should be removed after difference.
- remove_tool: bool = True - If the tool object should be removed after difference.

Returns: GeoObject

Raises: -

function em.geo.**add**()

Perform a boolean union between two GeoObjects. Must be of the same dimension.

Parameters

- main: GeoObject
- other: GeoObject
- remove_object: bool = True - If the main object should be removed after union.
- remove_tool: bool = True - If the tool object should be removed after union.

Returns: GeoObject

Raises: -

function em.geo.**intersect**()

Perform a boolean intersection between two GeoObjects. Must be of the same dimension.

Parameters

- main: GeoObject
- other: GeoObject
- remove_object: bool = True - If the main object should be removed after intersection.
- remove_tool: bool = True - If the tool object should be removed after intersection.

Returns: GeoObject

Raises: -

function em.geo.**embed**()

Embeds a 2D surface into a 3D volume such that the surface will be meshed without itself being a separate object.

Parameters

- main: GeoVolume
- other: GeoSurface

Returns: None

Raises: -

function em.geo.**rotate**()

Rotates the input geometry.

Parameters

- main: GeoObject
- co: tuple[float, float, float] - the rotation axis origin
- ax: tuple[float, float float] - The rotation axis
- angle: float - The rotation angle in degrees or radians
- make_copy: bool = False - If rotated object should be of a copy of the original or the original.
- degree: bool = True - If the rotation angle is given in degrees.

Returns: GeoObject

Raises: -

function em.geo.**translate**()

Translates the input geometry

Parameters

- main: GeoObject
- dx: float = 0.0
- dy: float = 0.0
- dz: float = 0.0
- make_copy: bool = False - If rotated object should be of a copy of the original or the original.

Returns: GeoObject

Raises: -

function em.geo.mirror()

Mirrors the input geometry

- Parameters
- main: GeoObject
 - origin: tuple[float, float, float] = (0,0,0) the mirror plane origin.
 - direction: tuple[float, float, float] = (0,0,1) the mirror plane normal axis.
 - make_copy: bool = True - If rotated object should be of a copy of the original or the original.
- Returns: GeoObject
- Raises: -

function em.geo.change_coordinate_system()

Transforms the input geometry from some coordinate system into another. The default starting and ending coordinate system is the global coordinate system.

- Parameters
- main: GeoObject
 - new_cs: em.Coordinatesystem = GCS - The coordinate system that the object is transformed into.
 - old_cs: em.CoordinateSystem = GCS - The coordinate system from which the object is transformed.
 - make_copy: bool = False - If rotated object should be of a copy of the original or the original.
- Returns: GeoObject
- Raises: -

function em.geo.stretch()

Stretches a geometry by some scalar ratio in the x, y and z axis relative to some origin

- Parameters
- main: GeoObject
 - fx: float = 1.0
 - fy: float = 1.0
 - fz: float = 1.0
 - origin: tuple[float, float, float] = (0,0,0)
- Returns: GeoObject
- Raises: -

function em.geo.extrude()

Extrudes an input GeoSurface object along some axis specified by dx, dy and dz

- Parameters
- main: GeoSurface
 - dx: float = 0.0
 - dy: float = 0.0
 - dz: float = 0.0
- Returns: GeoVolume
- Raises: -

function em.geo.unite()

Unite a sequence of GeoObjects of the same dimension into one object (similar to recursive unions). Example:

```
out = em.geo.unite(obj1, obj2, ... , objn)
```

- Parameters
- *objects: GeoObject -
- Returns: GeoObject
- Raises: -

function em.geo.stick()

Move some object such that the objects anchor point is placed at some other anchor point. See [Anchor](#). Example usage:

```
em.geo.stick(box1, box1.anch.Y, box2.anc.x)
```

See the manual for more details.

- Parameters
- object: GeoObject
 - p1: em.Anchor
 - p2: em.Anchor
- Returns: GeoObject
- Raises: -

function em.geo.bounding_box()

Returns the dimensions of a bounding box around all simulation domain objects.

- Parameters
- Returns: tuple[float,...] - xmin, xmax, ymin, ymax, zmin, zmax
- Raises: -

function em.geo.PCBLayer()

Creates a PCBLayer class object specifying a dielectric layer of a PCB layer stack.

- Parameters
- th: float - The layer thickness
 - mat: em.Material - The dielectric layer material
 - name: str - The layer name
- Returns: PCBLayer
- Raises: -

function em.geo.PCBNew()

Creates a PCBNew class object

- Parameters
- thickness: float - The thickness of the PCB in the pcb units.
 - unit: float = 1.0 - The unit (scaling constant) applied to all dimensions when building a PCB.
 - cs: CoordinateSystem = GCS - The coordinate system in which the PCB is placed
 - material: em.Material - The material of the PCB (dielectric)
 - trace_material: em.Material = PEC - The material of the PCB traces
 - layers: int = 2 - The number of conductor layers
 - stack: list[PCBLayer] - A list of PCBLayer objects defining the layer stack. Bypasses the thickness and material argument. See PCBLayer
 - trace_thickess: float - The thickness of the traces
 - zs: np.ndarray | None - An optional list at which the copper traces exist
 - thick_trackes: bool = False - If the traces are to be generated as 3D objects instead of flat 2D objects.
- Returns: PCBNew
- Raises: -

function em.geo.PCBNew.z()

Returns the Z-height of the copper trace layer specified by the number.

- Parameters
- int: float - The copper trace layer number from bottom to top.
- Returns: float
- Raises: -

function em.geo.PCBNew.add_vias()

Add vias at the specified coordinates. All arguments beyond the coordinates must be provided as keyword arguments. Example:

```
pcb.add_vias((0,0),(0,1),(2,0), ... ,radius=0.2, z1=pcb.z(0), z2=pcb.z(2))
```

- Parameters
- *Coordinates: tuple[float, float] - A sequence of (x,y) coordinates as arguments
 - radius: float - The via radius
 - z1: float - The starting height
 - z2: float - The ending height
 - segments: int = 6 - the number of polygon segments for the via.
- Returns: None
- Raises: -

function em.geo.PCBNew.load()

Returns a RouteElement object for the given label name.

- Parameters
- name: str
- Returns: RouteElement
- Raises: -

function em.geo.PCBNew.**determine_bounds()**

Establish the XY bounds of all the geometries in the PCB. Must be called after compile_paths.

- Parameters**
- leftmargin: float = 0
 - topmargin: float = 0
 - rightmargin: float = 0
 - bottommargin: float = 0

Returns: None

Raises: -

function em.geo.PCBNew.**set_bounds()**

Hard set the boundaries of the PCB

- Parameters**
- xmin: float
 - ymin: float
 - xmax: float
 - ymax: float

Returns: None

Raises: -

function em.geo.PCBNew.**plane()**

Creates a GeoSurface or GeoVolume plane object based on coordinate specifications relative to the PCB. If no dimensions and coordinates are provided, the entire bounds of the PCB will be used. Holes specified by vias will automatically be cut from these planes.

Can be used to generate a solid ground plane:

```
gnd = pcb.plane(pcb.z(0))
```

- Parameters**
- z: float - The Z-height of the plane
 - width: float | None - The plane width
 - height: float | none - The plane height
 - origin: tuple[float, float] | Anchor | None - The origin of the plane
 - alignment: em.CORNER | em.CENTER - The alignment point of the plane

Returns: None

Raises: -

function em.geo.PCBNew.**radial_stub()**

Add a radial stub geometry to the polygon list which will be generated.

- Parameters**
- post tuple[float, float] | Anchor - the position of the stub
 - length: float - The length of the radial stub
 - angle: float - The fan angle of the radial stub in degrees.
 - direction: tuple[float, float] - The direction of the radial stub
 - Nsections: int = 8 - The number of sections used to approximated the radial section.
 - wo: float = 0 - The starting width of the radial stub.
 - z: float = 0 - The Z-height of the radial stub.
 - material: em.Material - The material. Defaults to the trace material of the PCB.

Returns: None

Raises: -

function em.geo.PCBNew.**generate_pcb()**

Generates either a list of GeoVolume objects or a single GeoVolume object for each dielectric layer.

- Parameters**
- split_z: bool = True - Splits the Dielectric layer in multiple sections in case more than 2 copper layers are defined without a stack.
 - merge: bool = True - Merges the dielectric layers to one in case the dielectric material is always the same. (Undos split_z but is useful for PCB defined with layer stacks sometimes.)

Returns: list[GeoVolume] | GeoVolume

Raises: -

function em.geo.PCBNew.**generate_air()**

Creates an Air box exactly matching the PCB area above the PCB.

- Parameters**
- height: float - The Height of the air-box
 - bottom: bool = False - If the air box should be below instead of above the PCB.

Returns: GeoVolume

Raises: -

function em.geo.PCBNew.**new()**

Start a new StripPath sequence for routing PCB traces. See StripPath and the list of instructions on Section 3.7. for routing instruction specifications.

- Parameters**
- x: float | Anchor - The starting X coordinate
 - y: float - The starting Y coordinate (ignored when X is an anchor)
 - width: float - The trace width
 - direction: tuple[float, float] - The direction in which the trace will move forward.
 - z: float - The z-height of the trace

Returns: StripPath

Raises: -

function em.geo.PCBNew.**lumped_port()**

Create a lumped port surface at the specified location. The LumpedPort boundary condition information will be added as meta-data.

- Parameters**
- stripline: StripLine | str - The point where to place the lumped port or the label used when named during routing.
 - z_ground: float | None = None - The Z-height to which the lumped port is stretched. Defaults to the lowest layer.

Returns: GeoPolygon

Raises: -

function em.geo.PCBNew.**lumped_port_pts()**

Add the lumped port rectangle based on two coordinates that define a line segment which is extruded between two PCB layers.

- Parameters**
- p1: tuple[float, float] - The starting XY coordinate
 - p2: tuple[float, float] - The ending XY coordinate
 - z: float - The starting extrusion z-height
 - z_ground: float - The ending estrusion Z-height (defaults to the bottom layer).

Returns: GeoPolygon

Raises: -

function em.geo.PCBNew.**modal_port()**

Create a modal port surface to use for ModalPort boundary condition assignment.

- Parameters**
- point: StripLine | str - The point where to place the lumped port or the label used when named during routing.
 - height: float | tuple[float, float] -The height by which the modal port should reach above the trace. If a float is provided the bottom is on the bottom of the PCB. If its a tuple, the first height dimension will reach below the PCB.
 - width_multiplier: float = 5.0 - How much wider the modal port is than the trace itself
 - width: float | None - The fixed width of the modal port.

Returns: GeoPolygon

Raises: -

function em.geo.PCBNew.**generate_vias()**

Generates all the queued vias as a list of GeoVolume objects or single GeoVolume object..

- Parameters**
- merge: bool = False - If the vias should be combined into a single GeoVolume object.

Returns: list[GeoVolume] | GeoVolume

Raises: -

function em.geo.PCBNew.add_poly()

Add an arbitrary polygon to the queue of objects to be created.

Parameters

- xs: list[float] - List of x-coordinates
- ys: list[float] - List of Y-coordinates
- z: float - The Z-height of the polygon (defaults to the top layer)
- material: em.Material - The material of the polygon. Defaults to the trace material.

Returns: None

Raises: -

function em.geo.PCBNew.add_hole()

Add a hole to be cut form all polygon objects.

Parameters

- xs: list[float] - A list of x-coordinates
- ys: list[float] - A list of Y-coordinates
- z: float - The z-coordinate. Defaults to the top layer.
- material

Returns: None

Raises: -

function em.geo.PCBNew.compile_paths()

Compile paths will turn all queued strip paths and polygons into a list of GeoPolygon objets or GeoVolume (if thick traces is turned on).

Parameters

- merge: bool = False - If a boolean union should be performed and subsequent merger of all geometries into the same GeoObject

Returns: list[GeoVolume|GeoSurface] | GeoVolume | GeoSurface

Raises: -

property em.geo.StripPath.xs/ys

A list of all x/y-coordinates.

Returns: list[float]

property em.geo.StripPath.start

The first RouteElement object

Returns: RouteElement

property em.geo.StripPath.end

The last RouteElement object

Returns: RouteElement

property em.geo.StripPath.bck

Moves the last RouteElement one half stripline width backwards and return this object.

Returns: StripPath

property em.geo.StripPath.skip

Prepares to remove half of the stripline width in distance from the next route element. Return this object.

Returns: StripPath

function em.geo.StripPath.straight()

Move the StripPath a fixed distance forward in the current route direction.

Parameters

- distance: float - The line distance
- width: float - the width of this line segment. (Defaults to the current width)
- dx: float | None - An optional x-offset for where the routing should start (edges will be connected.)
- dy: float | None - An optional y-offset for where the routing should start.

Returns: StripPath

Raises: -

function em.geo.StripPath.taper()

Move some distance forward and taper the line width to a new width

Parameters

- distance: float - The line distance
- width: float - The new width to taper to

Returns: StripPath

Raises: -

function em.geo.StripPath.turn()

Perform a turn at this location. The rotation point will be either the right or left corner of the current routing position. Not the heartline. Use `pturn` for the heartline.

Parameters

- angle: float - The turn angle in degrees (negative for opposite directions. Positive is clockwise)
- width: float - The width at which to start the turn.
- corner_type: “champher” | “square” = “square” - The type of turn. Square is a straight corner. Champher cuts off the corner.
- dsratio: float = 0.7 - The champher distance into the line width expressed as a ratio of the width of the trace.

Returns: StripPath

Raises: -

function em.geo.StripPath.pturn()

Same as the turn but the rotation point is at the center of the trace. This will consume some distance of line from the last and next routing section. **can cause invalid traces when routed to closely to other turns.**

Parameters

- angle: float - The turn angle in degrees (negative for opposite directions. Positive is clockwise)
- corner_type: “champher” | “square” = “square” - The type of turn. Square is a straight corner. Champher cuts off the corner.
- dsratio: float = 0.7 - The champher distance into the line width expressed as a ratio of the width of the trace.

Returns: StripPath

Raises: -

function em.geo.StripPath.curve()

Route a curved path with some circle radius. The in and outside line segments are approximated by straight line sections.

Parameters

- angle: float - The curve angle (degrees)
- radius: float - The curvature radius
- width: float | None - The line width at which to route the curve
- dang: float = 10.0 - The angular interval with which to segment the curve (degrees).

Returns: StripPath

Raises: -

function em.geo.StripPath.store()

Store the current position (RouteElement) in the PCBNew memory to be recalled for XY coordinates later using `load`. Also see `lumped_port` and `modal_port`

Parameters

- name: str | int - Any hashable code or identifier.

Returns: StripPath

Raises: -

function em.geo.StripPath.split()

Store the current position and start a completely new trace at the current location. Can later be returned to using `merge`

Parameters

- direction: tuple[float, float] | None - an optional new direction for the new trace
- angle: float | None - A new angle at which to depart (instead of direction)
- width: float | none - The new trace width

Returns: StripPath

Raises: -

function em.geo.StripPath.**lumped_element()**

Add a lumped_element surface polygon at the current position and move ahead to where the element stops.

- Parameters**
- impedance_function: Callable - The function $Z(f)$ defining the impedance of the component.
 - size: “0402” etc | tuple[float, float] - The width and height of the component. Accepts the common SMD component sizes as strings.
- Returns:** StripPath
- Raises:** -

function em.geo.StripPath.**stub()**

Add a signal rectangular stub in some direction allowing you to continue as you where going. Stubs are placed starting from the center of the stripline, not the side edge.

- Parameters**
- direction: tuple[float, float] - The direction in which the stub is layed out
 - width: float - The width of the stub
 - length: float - The length of the stub
 - mirror: bool = False - If the stub should also be added on the opposite side (in the opposite direction).
- Returns:** StripPath
- Raises:** -

function em.geo.StripPath.**merge()**

Move the current routing position back to where `split` was called last

- Parameters**
- Returns:** StripPath
- Raises:** -

function em.geo.StripPath.**via()**

Add a via in the current position to a new layer.

- Parameters**
- znew: float - The new z-height of where the via stops
 - radius: float - The via radius
 - proceed: bool = True - True if you wish the routing to continue from where the via is routing to. Set to false if you wish to proceed on the current layer.
 - direction: tuple[float,float] | None - An optional new direction for the new tarce to head to.
 - width: float | None - The optional new trace width
 - extra: float | None - How much extra trace material to add after the current via position. Defaults to half the trace width.
 - segments: int = 6 - The number of polygon segments for the via cross-section.
 - hole_radius: float | None - The radius of the hole which will be cutout from all ground planes.
 - hole_skip_layers: list[int] | None - Which layer numbers **not** to include when cutting out holes (In case you wish for the via to make contact with them).
 - reverse: float = 0.0 - Some distance to move the via back relative to the current route direction.
- Returns:** StripPath
- Raises:** -

function em.geo.StripPath.**short()**

Add a short circuit via to some ground layer.

- Parameters**
- ground_layer: int = 0 - The ground layer to connect to
 - radius: float - The via radius
 - reverse: float = 0.0 - How much to place the via backwards relative to the current route direction.
- Returns:** StripPath
- Raises:** -

function em.geo.StripPath.**jump()**

Start a new trace by jumping relative to the current position. The jump displacement can be provided in two ways:

- dx + dy
- gap + side + reverse (useful for coupled line sections)

- Parameters**
- dx: float | None = None - The x-displacement
 - dy: float | None = None - The y-displacement
 - width: float | None = None - The trace width of the new line
 - direction: tuple[float, float] | None = None - The new movement direction of the new trace
 - gap: float | None - The gap to leave when jumping sides (right or left)
 - side: “left” | “right” - The direction to jump.
 - reverse: float | None = None - Distance to jump backwards relative to the current route direction.
- Returns:** StripPath
- Raises:** -

function em.geo.StripPath.**to()**

Does a straight forward auto-route to a destination location. The route tries to find a valid turn angle to point towards the intended “intercept point” and then arrives at the destination based on the arrival direction and distance.

- Parameters**
- dest: tuple[float, float] - The destination coordinate
 - arrival_dir: tuple[float, float] - The arrival direction.
 - arrival_margin: float | None = None - How much straight line segment you wish to add before you arrive at the destination (arrival_dir)
 - angle_step: float = 90 - Which angular steps are allowed when figuring out the arrival point.
- Returns:** StripPath
- Raises:** RuntimeError: If no route can be found. Try chaning the angle_step if this occurs to some lower value.

9.1 Meshing Functions

function em.Simulation.**set_resolution()**

Specifies the maximum mesh size as a fraction of th greatest simulation wavelength. A factor of 0.3 means that the mesh in any domain may not be larger than $0.3 \frac{c_0}{f_{\max}}$. See also `set_resolution#mw`

- Parameters**
- resolution: float - The max mesh size resolution.
- Returns:** None
- Raises:** -

function em.Mesher.**set_curved_boundary_meshing()**

Specify a detail factor by which to approximate curved boundaries. A higher number means that curved boundaries are meshed with more elements.

- Parameters**
- factor: float = 10.0 - The detail factor
- Returns:** None
- Raises:** -

function em.Mesher.**unset_constraints()**

Remove all mesh size constraints from a geometric entity specified by a list of dimtag pairs.

- Parameters**
- dimtags: list[tuple[float, float]] - A list of dimtag tuples.
- Returns:** None
- Raises:** -

function em.Mesher.**set_boundary_size()**

Specify a mesh size on the boundary of a given geometric entity. The boundary of a geometry is always one dimension lower than the geometry itself. The growth-rate represents how much the mesh size is allowed to grow away from where its specified.

- 1.0 - No growth
- 2.0 - Each next element away from the last can be twice larger.

At some point a larger growth rate has very little effect.

- Parameters**
- boundary: GeoObject | Selection | Iterable - A the geometries of which you want the boundary to be meshed finer.
 - size: float - The desired mesh size on the boundary
 - growth_rate: float = 3.0 - How fast the mesh size is allowed to grow away from the boundary
 - max_size: float | None = None - The maximum mesh size away from the boundary. Defaults to the max mesh-size specified by the material + resolution.

Returns: None

Raises: -

function em.Mesher.**set_trace_refinement()**

Apply a trace refinement mesh size specification for a set of 2D polygons in the same plane. The mesh size is determined based on how close parallel lines are with respect to each other. The trace refinement algorithm requires these line segments to overlap some minimum amount before they are counted as being parallel.

- Parameters**
- obj: GeoSurface | FaceSelection | Iterable - Which surfaces to apply this algorithm to
 - quality_factor: float = 3.0 - The reduction maximum mesh size relative to the distance between lines. A factor of 3.0 means the mesh size has to be 3.0 times smaller than the distance.
 - growth_rate: float = 3.0 - How fast the mesh size may increase away from the refined locations
 - min_size: float = 1e-5 - The minimum allowed mesh size
 - max_size: float | None - The maximum mesh size. Defaults to the max mesh size specified by the model resolution.
 - min_overlap: float = 0.75 - How much two lines must overlap when projected onto each other before the refinement may count. (Prevents lines that barely overlap to be refined entirely.). 0.75 means 75% overlap in length.

Returns: None

Raises: -

function em.Mesher.**set_domain_size()**

Specify the maximum size in a domain

- Parameters**
- obj: GeoObject | Selection
 - size: float - The maximum size

Returns: None

Raises: -

function em.Mesher.**set_face_size()**

Specify the maximum size on a surface

- Parameters**
- obj: GeoSurface | Selection
 - size: float - The maximum size

Returns: None

Raises: -

function em.Mesher.**set_pec_face()**

Specifies that a face can be meshed as coarsely as is allowed. Can be used to save mesh elements on PEC faces where the E-field is 0 anyway.

- Parameters**
- obj: GeoSurface | Selection
 - size: float | None - An optional maximum mesh size.

Returns: None

Raises: -

function em.Mesher.**set_size()**

Specify the maximum size in an object of any dimension.

- Parameters**
- obj: GeoObject
 - size: float - The maximum size

Returns: None

Raises: -

9.2 Microwave Physics

function Microwave3D.**reset()**

Resets the boundary conditions and microwave physics state variables.

- Parameters**
- _reset_bc: bool = True - If the boundary conditions should be reset.

Returns: None

Raises: -

function Microwave3D.**set_frequency()**

Set a single frequency or a list of frequencies as simulation frequencies.

- Parameters**
- frequency: float | Iterable[float] - The frequency or frequencies.

Returns: None

Raises: -

function Microwave3D.**set_frequency_range()**

Specify the frequency range as a `np.linspace` like format.

- Parameters**
- fmin: float - The minimum frequency
 - fmax: float - The maximum frequency
 - Npoints: int - The number of points

Returns: None

Raises: -

function Microwave3D.**set_resolution()**

Specifies the maximum mesh size as a fraction of the greatest simulation wavelength. A factor of 0.3 means that the mesh in any domain may not be larger than $0.3 \frac{c_0}{f_{\max}}$

- Parameters**
- resolution: float - The resolution as a fraction of the wavelength.

Returns: None

Raises: -

function Microwave3D.**modal_analysis()**

Perform a port eigenmode analysis to find the eigenmodes of a ModalPort object. Modal solutions are stored inside the port object.

- Parameters**
- port: ModalPort - The port object
 - nmodes: int = 6 - The number of modes to look for
 - direct: bool = True - If a direct eigenmode solver should be used. These solvers become significantly slower above 1000 degrees of freedom.
 - TEM: bool = False - If the port should assume that the eigenmode solutions are TEM modes
 - target_kz: float | None = None - An estimate for the out-of-plane propagation constant. Used to accelerate the search with non-direct solvers
 - target_neff: float | None = None - An alternative expression of the expected out of plane propagation constant but using an effective mode index defined such that $k_z = n_{\text{eff}} k_0$.
 - freq: float | None = None - The frequency for which you would wish to compute the eigenmode.

Returns: None

Raises: -

function Microwave3D.run_sweep()

Perform a frequency sweep simulation. If one frequency is specified, you also use this function.

Parameters

- parallel: bool = False - If you wish to distribute the solve process of different frequencies across multiple cores.
- n_workers: int = 2 - How many different worker processes or threads to use when executing a parallel solve.
- harddisc_path: str = "EMergeSparse" - A storage sub-directory name to temporarily store the sparse matrices before solving to save RAM memory. Only used if `cache_hddisk = True`.
- frequency_groups: int = -1 - The number of frequencies to pre-assemble before solving. If the value is -1, all frequencies will be pre-assembled. For large frequency sweeps this can consume a significant amount of RAM memory.
- cache_hddisk: bool = False - If the sparse matrices to be solved should be stored on the hddrive after assembly and loaded before solving. Saves RAM memory but incurs a slight penalty. Usually not more useful that setting the frequency groups to 1. It does also store all the solutions after solving so it still can save some RAM.
- multi_processing: bool = False - If Multi-processing should be used during distributed sweeps over multi-threading. Required when using the UMFPACK, Apple Accellerate and Cholesky spase solvers. Requires entry point protection.
- automatic_modal_analysis: bool = True - If the modal analysis of the ModalPorts should be automatically executed when solving.
- _reset_solvers: bool = True - If the solver states should be reset after solving. Should almost always be set to True.

Returns: MWData

Raises: -

function Microwave3D.run_adaptive_sweep()

Broken inefficient version of run_sweep to solve for just enought points to have vector-fitting fill in the blanks. Doe snot work correctly yet.

Parameters

Returns: MWData

Raises: -

function Microwave3D.run_scattered()

Performs a scattered field simulation (Requires the existence of the ScatteredField boundary condition. See [ScatteredField](#)).

Parameters

- parallel: bool = False - If you wish to distribute the solve process of different frequencies across multiple cores.
- n_workers: int = 2 - How many different worker processes or threads to use when executing a parallel solve.
- harddisc_path: str = "EMergeSparse" - A storage sub-directory name to temporarily store the sparse matrices before solving to save RAM memory. Only used if `cache_hddisk = True`.
- frequency_groups: int = -1 - The number of frequencies to pre-assemble before solving. If the value is -1, all frequencies will be pre-assembled. For large frequency sweeps this can consume a significant amount of RAM memory.
- cache_hddisk: bool = False - If the sparse matrices to be solved should be stored on the hddrive after assembly and loaded before solving. Saves RAM memory but incurs a slight penalty. Usually not more useful that setting the frequency groups to 1. It does also store all the solutions after solving so it still can save some RAM.
- multi_processing: bool = False - If Multi-processing should be used during distributed sweeps over multi-threading. Required when using the UMFPACK, Apple Accellerate and Cholesky spase solvers. Requires entry point protection.
- automatic_modal_analysis: bool = True - If the modal analysis of the ModalPorts should be automatically executed when solving.

Returns: MWData

Raises: -

function Microwave3D.eigenmode()

Perform an eigenmode simulation on the simulation domain.

Parameters

- search_frequency: float - The frequency to search for eigenmodes around
- nmodes: int = 6 - The number of solutions to look for.
- ko_limit: float = 1.0 - A lower limit for the solution phase constant used to ignore null-space modes.
- direct: bool = False - If a direct solver should be used. Setting direct to true turns the A and B matrices into dense matrices which will explode the RAM and CPU time for problems with more than 1000 Degrees of Freedom.
- mode: "LM" | "LR" | "SR" | "LI" | "SI" = "LM" - The search mode for ARPACK. Tries to find eigenmodes with eigenvalues that have:
 - LM = Largest Magnitude
 - LR = Largest Real part
 - SR = Smallest Real part
 - LI = Largest Imaginary part
 - SI - Smallest Imaginary Part

Returns: MWData

Raises: -

9.3 Microwave Boundary-Conditions

The following are available under your simulation object:

python

```
1 sim = em.Simulation(...)
2 sim.mw.bc.PEC(...) # For example
```

function MWBoundaryConditionSet.PEC()

Add a perfect-electric-conductor boundary condition.

Parameters

face: FaceSelection | GeoSurface

Returns: BoundaryCondition

Raises: -

function MWBoundaryConditionSet.PMC()

Add a perfect-magnetic-conductor boundary condition.

Parameters

face: FaceSelection | GeoSurface

Returns: BoundaryCondition

Raises: -

function MWBoundaryConditionSet.AbsorbingBoundary()

Adds an absorbing boundary condition(ABC). This is also called the Radiation boundary condition in many solvers. It can be the first order ABC or second order (ABC). The second order can absorb radiation better from larger angles of incidence. (See manual). The ABC type for order changes at what angle of incidence range the second order correction is optimized. See the graph below.

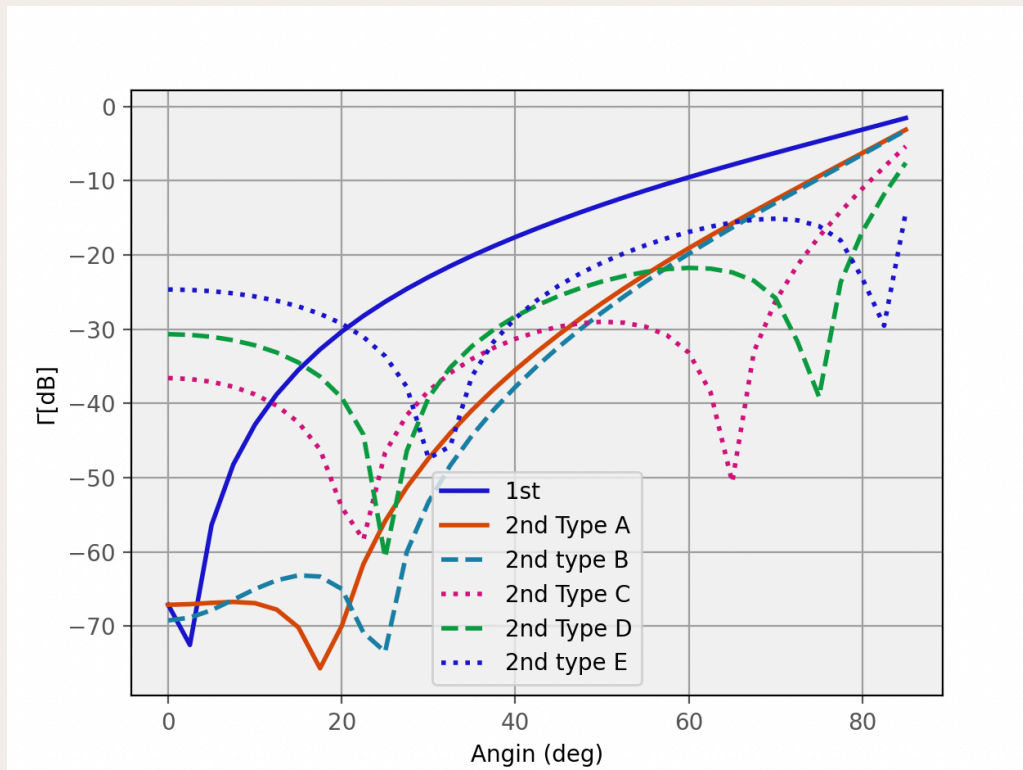


Figure 1: The reflection coefficient of the different ABC types.

Parameters

- face: FaceSelection | GeoSurface
- order: int = 1 - Either 1 for first order or 2 for second order.
- abctype: "A" | "B" | "C" | "D" | "E" = "B" - A type label for the correction factor. From A to E, A improves absorption for lower angles of incidence. "E" Has better performance at very large angles of incidence but decreases the effectiveness at lower angles of incidence.

Returns: BoundaryCondition

Raises: -

function MWBoundaryConditionSet.ScatteredField()

Adds a ScatteredField boundary condition. This boundary condition should be applied to a surface enclosing the entire simulation domain. The ScatteredField boundary condition is used exclusively for electromagnetic scattering simulations such as radar-cross-section computations.

Parameters

- face: FaceSelection | GeoSurface
- power_density: float = $1/(2 \cdot Z_0)$ - The plane wave power density. Defaults to a $1Wm^{-2}$
- cs: em.CoordinateSystem - The coordinate system for the plane wave. The plane wave has zero phase at the coordinate system origin. Defaults to the global coordinate system.

Returns: BoundaryCondition

Raises: -

function ScatteredField.set_excitations()

Define the set of plane wave excitations for the ScatteredField boundary condition. The excitations use an elevation/azimuth coordinate system definition:

- $\theta(deg)$ - elevation
- $\varphi(deg)$ - azimuth
- $\psi(deg)$ - polarization

Then:

- $\theta = 0, \varphi = 0, \psi = 0: \mathbf{k} = +X, \mathbf{E} = E_z \hat{z}$
- $\theta = 0, \varphi = 90, \psi = 0: \mathbf{k} = +Y, \mathbf{E} = E_z \hat{z}$
- $\theta = 90, \varphi = 0, \psi = 0: \mathbf{k} = -Z, \mathbf{E} = E_x \hat{x}$
- $\theta = 0, \varphi = 0, \psi = 90: \mathbf{k} = +X, \mathbf{E} = E_y \hat{y}$

Please check the manual for more details.

Parameters

- thetas: np.ndarray | float = 0.0 - One or more theta angles
- phis: np.ndarray | float = 0.0 - One or more phi angles
- polarizations: np.ndarray | float = 0.0 - One or more psi (polarization) angles.

Returns: None

Raises: -

function MWBoundaryConditionSet.LumpedElement()

Adds a lumped element boundary condition to a surface. This boundary condition only holds true results if the surface you assign it to is isometric to a rectangle (so rectangles, boundaries of cylinders etc.)

Parameters

- face: FaceSelection | GeoSurface
- impedance_function: Callable | None = None - A callable as a function of frequency that returns the impedance $Z(f)$. This function must be defined unless you pass a **lumped impedance surface** object created by the **PCBNew** class.

Returns: BoundaryCondition

Raises: -

function MWBoundaryConditionSet.SurfaceImpedance()

Adds a SurfaceImpedance boundary condition. The surface impedance boundary condition must be used to model the boundaries of conductors. It accurately models the skin effect. **Do not use the SurfaceImpedance boundary condition to model 2D geometries that represent traces with finite thickness such as (micro-)striplines. Use the ThinConductor boundary condition instead.**

This boundary condition can be used to model for example the losses on the walls of a waveguide or the boundary surface of a coaxial core conductor.

Parameters

- face: FaceSelection | GeoSurface
- material: em.Material | None = None - The material of the surface. Must be defined or a specific surface conductance value
- surface_conductance: float | None - The surface conductance in $\frac{S}{m^2}$
- surface_roughness: float = 0.0 - Surface roughness parameter in meters RMS used in the Hammerstad-Jensen model.
- thickness: float | None = None - The thickness of this specific material.
- impedance_function: Callable | None = None - An optional impedance function instead of the automatic calculation. The impedance function $Z(f)$ must return what would be the wave-impedance if it was representing vacuum.

Returns: BoundaryCondition

Raises: -

function MWBoundaryConditionSet.ThinConductor()

Adds a ThinConductor boundary condition. The thin impedance boundary condition must be used to model thin conducting object that you wish to model as a 2D surface instead. ThinConductor boundaries must not border on a bulk conductor. It must be assigned to flat structures like PCB traces etc.

Parameters

- face: FaceSelection | GeoSurface
- material: em.Material | None = None - The material of the surface. Must be defined or a specific surface conductance value
- surface_conductance: float | None - The surface conductance in $\frac{S}{m^2}$
- surface_roughness: float = 0.0 - Surface roughness parameter in meters RMS used in the Hammerstad-Jensen model.
- thickness: float | None = None - The thickness of this specific material.
- impedance_function: Callable | None = None - An optional impedance function instead of the automatic calculation. The impedance function $Z(f)$ must return what would be the wave-impedance if it was representing vacuum.

Returns: BoundaryCondition

Raises: -

function MWBoundaryConditionSet.ModalPort()

Adds a ModalPort object to the simulation. A modal port is also called a “Wave Port” or “Numeric Port” in other solvers. Modal ports require a modal field solution to first be computed. This is done automatically when you call `.run_sweep()` or can be done manually with the `.modal_analysis()` function of the `Microwave3D` class.

- Parameters
- face: FaceSelection | GeoSurface
 - port_number: int - The port number. A simulation with N ports must have all ports numbered 1 to N.
 - modetype: “TEM” | “TE” | “TM” - The mode type you would like the solver to focus on. Modes of another type will be ignored in case multiple can exist. Quasi-TEM modes are labeled as “TEM” as well.
 - number_of_modes: int = 1 - The number of modes you wish to include in the solver. Keep at 1 unless you have a port that can support multiple modes that have the same propagation constant (such as circular or quare waveguides).
 - mixed_materials: bool = False - Set to true if the port consists of multiple different materials that will cause quasi-TEM modes to be imposed. This will cause EMerge to re-compute the port mode field and propagation constant for each frequency. Required to accurately model microstriplines but at low frequencies less important.
 - impedance_definition: “PV” | “PI” | “VI” - The impedance definition to use for the port. Requires voltage and current integration lines. For ports with a well defined impedance (TEM or quasi-TEM modes) with two separated conductor groups, these lines will be automatically generated. You can also define them manually.
 - power: float = 1.0 - The port power. Keep at 1W
 - cs: em.CoordinateSystem - An optional coordinate system for the port. Never needed in practice.

Returns: BoundaryCondition
Raises: -

function ModalPort.set_integration_line()

Define the integration line for the voltage between two points in 3D space.

- Parameters
- c1: tuple[float, float, float] - The start of the integration line
 - c2: tuple[float, float, float] - The end of the integration line
 - N: int = 51 - The number of points along the integration line.

Returns: None
Raises: -

function ModalPort.align_modes()

Define a sequence of Axis objects or vectors as tuples that are used in order to align/orient the modal solutions. This can be used with square or circular waveguides for example to make sure that the two fundamental modes are ordered consistently and point in the same direction at each frequency.

This will cause the order to be kept consistent and it will reverse the polarization so that the field at phase=0 is always pointing in the same direction.

Example: You have a square waveguide in the XZ plane and you want to make sure that mode 1 points in the +X direction and mode 2 in the +Z direction then you can do:

python

```
1 mp_object.align_modes(em.XAX, em.ZAX)
2 # or
3 mp_object.align_modes((1,0,0), (0,0,1))
```

- Parameters
- *axes: tuple | np.ndarray | Axis - A sequence of vectors/axes to align the modes with.
- Returns: None
Raises: -

function ModalPort.set_terminals()

Label geometries as terminals in case you have a modal port that consists of 3 or more separate conductors such as differential microstriplines. The two terminals will be connected straight with a voltage integration line.

- Parameters
- positive: Selection | GeoObject | None = None - The positive terminal.
 - negative: Selection | GeoObject | None = None - The negative terminal.
 - ground: **unused currently**
- Returns: None
Raises: -

function MWBoundaryConditionSet.FloquetPort()

Creates a FloquetPort boundary condition. If you used `RectCell` or `HexCell` to define the periodicity of your simulation domain use `floquet_port` instead.

- Parameters
- face: FaceSelection | GeoSurface
 - port_number: int - The port number. A simulation with N ports must have all ports numbered 1 to N.
 - cs: em.CoordinateSystem
 - power: float = 1.0 - The port power
 - er: float = 1.0 - The dielectric constant that the port finds itself in.
- Returns: BoundaryCondition
Raises: -

function MWBoundaryConditionSet.floquet_port()

Adds a floquet port boundary condition.

- Parameters
- poly: GeoSurface - The floquet port object created from the `HexCell` or `RectCell` classes (See manual).
 - port_number: int - The port number. A simulation with N ports must have all ports numbered 1 to N.
- Returns: BoundaryCondition
Raises: -

function MWBoundaryConditionSet.RectangularWaveguide()

Creates a RectangularWaveguide boundary condition. The Rectangular-Waveguide boundary condition must be applied to surfaces that are rectangular.

- Parameters
- face: FaceSelection | GeoSurface
 - port_number: int - The port number. A simulation with N ports must have all ports numbered 1 to N.
 - mode: tuple[int, int] = (1, 0) - The Mode to excite. Currently only $TE_{n,m}$ modes are supported.
 - er: float = 1.0 - The dielectric constant of the material the port finds itself in.
 - cs: CoordinateSystem | None = None - The local coordinate system. Will be automatically inferred
 - dims: tuple[float, float] | None = None - The dimensions of the port (width, height). Are also automatically inferred. Only pass these if the inferred values are incorrect.
 - power: float = 1.0 - The port power. Keep at 1W.
- Returns: BoundaryCondition
Raises: -

function RectangularWaveguide.align_modes()

Define vector using a tuple or numpy array or Axis objects that will be used used to align the modes in the right direction. For the fundamental TE₁₀ mode, the phase=0 polarization may point either up or down which an add arbitrary phase shifts. By setting this vector you can make the port phase consistent.

Example: You have a rectangular waveguide in the XZ plane and you want to make sure that mode points up in the +Z direction use:

python

```
1 mp_object.align_modes(em.ZAX, )
2 # or
3 mp_object.align_modes((0,0,1))
```

- Parameters
- axis: tuple | np.ndarray | Axis - The alignment vector
- Returns: None
Raises: -

function MWBoundaryConditionSet.CoaxPort()

Creates a CoaxPort boundary condition. The coax port only supports the fundamental TEM coax mode. It is usually less accurate than that of a ModalPort because the curved boundaries of a coax port are rarely so ideal that the analytical solution is sufficiently accurate. Differences can be small. This will save a lot of simulation time in some cases.

Parameters	- face: FaceSelection GeoSurface - port_number: int - The port number. A simulation with N ports must have all ports numbered 1 to N. - rad_in_out: tuple[float, float] - The inner and outer radius of the coaxial port. - cs: CoordinateSystem - The local coordinate system of the coaxial port. This must have the origin exactly at the center of the port and the XY axes in plane of the port. these are not inferred automatically yet. - er: float = 1.0 - The dielectric constant of the coaxial port - power: float = 1.0 - The port power. Keep at 1W
Returns:	BoundaryCondition
Raises:	-

function MWBoundaryConditionSet.UserDefinedPort()

Create A UserDefinedPort object with a manual E-field function definition.

Since version 2.7, this port may have issues with TEM like non plane wave ports because EMerge requires the magnetic field definition as well for the characteristic impedance calculation. It currently does not finite differences for example to compute the curl.

Parameters	- face: FaceSelection GeoSurface - port_number: int - The port number. A simulation with N ports must have all ports numbered 1 to N. - Ex: Callable None = The electric field function $E_{x(k_0,x,y,z)}$ where x,y and z are provided in the local coordinate system. Defaults to $E_{x(k_0,x,y,z)} = 0$ - Ey: Callable None = The electric field function $E_{y(k_0,x,y,z)}$ where x,y and z are provided in the local coordinate system. Defaults to $E_{y(k_0,x,y,z)} = 0$ - Ez: Callable None = The electric field function $E_{z(k_0,x,y,z)}$ where x,y and z are provided in the local coordinate system. Defaults to $E_{z(k_0,x,y,z)} = 0$ - kz: Callable None = The out of plane propagation constant function $k_{z(k_0)}$ defaults to $k_{z(k_0)} = k_0$ - power: float = 1.0 - The power in Watts. Keep at 1.0 - modetype: "TEM" "TE" "TM" - The port mode type - cs: em.CoordinateSystem - The local port coordinate system. Defaults to the global coordinate system GCS.
Returns:	BoundaryCondition
Raises:	-

function MWBoundaryConditionSet.LumpedPort()

Create A LumpedPort object in the simulation domain. Lumped ports are ought to be used on small surfaces that are isometric to a rectangle (so rectangles and the boundary of a cylinder etc.). It works best if the port is small relative to the wavelength. The port mode will be a uniform E-field. The lumped port will become a sheet resistor that has a total resistance equal to the port impedance.

You must provide EMerge with information of the port dimensions and the direction of the E-field. If you pass the geometry defined by the `PCBNew` class's `lumped_port` function, these properties will be passed as metadata automatically.

Parameters	- face: FaceSelection GeoSurface - port_number: int - The port number. A simulation with N ports must have all ports numbered 1 to N. - width: float None = None - The width of the port (dimension orthogonal to the E-field polarization) - height: float None = None - The height of the port (dimension parallel to the E-field polarization) - direction: tuple[float, float, float] Axis - The E-field polarization direction. This vector usually points along the direction between two conductors (from one conductor to another). - power: float = 1.0 - The port power, keep at 1W - Zo: float = 50.0 - The port impedance. Defaults to 50Ω. This impedance will be literally added inside the simulation domain.
Returns:	BoundaryCondition
Raises:	-

10 Data

function em.MWData()

A generic dataclass container for all microwave related data. Has two main properties:

- `.scalar` - All scalar data like S-parameters and port impedances.
- `.field` - Stores all field solutions

Parameters	Should not be created manually by users.
Returns:	None
Raises:	-

function em.MWData.merge_with()

Merge all solutions of this dataset with other MWData objects.

Parameters	*others: MWData - Other dataset objects as a sequence of arguments.
Returns:	MWData
Raises:	-

function em.MWData.export_farfields()

Merge all solutions of this dataset with other MWData objects.

Parameters	- filename: str - The filename to export to - face: FaceSelection GeoSurface - The integration surface for the farfield calculation. - thetas: np.ndarray - An array of theta angles - phis: np.ndarray - An array of phi angles - origin: tuple[float, float, float] None, optional - An optional array for a radiation origin used to determine the normal vectors of farfield boundaries if inside vs. outside is not well defined. Defaults to None. - syms: list["Ex","Ey","Ez","Hx","Hy","Hz"] None - An optional set of simulation domain symmetries. Defaults to None. - precision: int = 4 - The number of decimals for the output file. Defaults to 4. - frequencies: list[float] None - The frequencies to pick for the output. Defaults to None.
Returns:	None
Raises:	-

function em.MWField()

A class containing a set of EM field solutions for each excited port for a single frequency. The solution is stored as the solution vector to the original FEM problem: $\mathbf{Ax} = \mathbf{b}$. To compute the E-field at specific coordinates you must use the `interpolate` function to compute the field at those coordinates.

Parameters	Should not be created manually by users.
Returns:	None
Raises:	-

property em.MWField.relative

A method chaining property that flips the fields to the relative field instead of the total field. Usable with fields generated by the scattered field simulation. Example:

```
python
1 data = sim.mw.scattered_field()
2 field = data.field.find(freq=1e9).relative.some_function()
```

Returns:	MWField
-----------------	---------

function em.MWField.excite_port()

Set the port excitation of a single port number with a specific complex amplitude.

Parameters	- number: int float = The port (port mode) index of the port(mode) to be excited. - excitation: complex = 1 + oj - The complex port excitation.
Returns:	None
Raises:	-

function em.MWField.**set_excitations()**

Specify the port excitation of each port in the dataset as a sequence of complex amplitude arguments. Example usage:

python

```
1 field.set_excitations(0,1.0,0) # three port simulation
```

Parameters • *excitations: complex - A sequence of arguments for each port(mode) excitation in order.
Returns: None
Raises: -

function em.MWField.**combine_ports()**

Combines two ports to function as a differential port where the two port indices i and j are replaced respectively by:

- $i \rightarrow i + j$ differential mode
- $j \rightarrow i + j$ common mode

Parameters • p1: int - The first port mode number.
• p2: int - The second port mode number.
Returns: MWfield
Raises: -

function em.MWField.**interpolate()**

Compute the E-field and H-field at the requested coordinates and return an MWField data container object. See [EHField](#)

The data can be passed as a numpy array of any dimensionality. Data will be returned in the same dimension.

Parameters • xs: np.ndarray - The X-coordinates as a numpy array.
• ys: np.ndarray - The Y-coordinates as a numpy array.
• zs: np.ndarray - The Z-coordinates as a numpy array.
• use nan: bool = True - If true, all points outside the simulation domain will be NaN. Otherwise they will be 0.
Returns: MWfield
Raises: -

function em.MWField.**int_surf()**

Computes a surface integration along a surface object or face selection.

To specify which field to integrate, the user must provide a function that takes a single argument (EHField object) and returns a specific field component. Example

python

```
1 field.int_surf(selection, lambda eh: eh.Sx)
```

Currently normal vectors are not available quantities which means that flux integrations of curved boundaries is difficult.

Parameters • surface: FaceSelection | GeoSurface - The surface to integrate
• argument: Callable - A lambda function that takes an EHField object and returns a field component to integrate.
• gqo: int = 4 - The Gauss Quadrature integration order.
Returns: complex | float
Raises: -

function em.MWField.**int_vol()**

Computes a volume inside a given volume.

To specify which field to integrate, the user must provide a function that takes a single argument (EHField object) and returns a specific field component. Example

python

```
1 field.int_vol(selection, lambda eh: eh.Qv)
```

Parameters • domain: DomainSelection | GeoVolume - The domain to integrate over.
• argument: Callable - A lambda function that takes an EHField object and returns a field component to integrate.
• gqo: int = 4 - The Gauss Quadrature integration order.
Returns: complex | float
Raises: -

function em.MWField.**int_line()**

Computes a line integral along a [Line](#) object.

Generally less practical than using the Line objects own line integral function. See [line_integral](#).

To specify which field to integrate, the user must provide a function that takes a single argument (EHField object) and returns a specific field component. Example

python

```
1 field.int_line(lineobj, lambda eh: eh.Hx)
```

Parameters • line: Line - The line object to integrate
• argument: Callable - A lambda function that takes an EHField object and returns a field component to integrate.
Returns: complex | float
Raises: -

function em.MWField.**int_lumped_element()**

Performs an averaged line integration of either the E, H or S field along the lumped element to compute for example the lumped element voltage (E). Currently it always takes the integral dot dl, so integrating H does not return the current (not really a useful quantity).

Parameters • lumped_element: LumpedElement - The lumped element boundary condition object. See [LumpedElement](#).
• axis: Axis | tuple[float, float, float] - An integration direction axis. This is not available in the LumpedElement itself.
• quantity: “E”, “H”, “S” = “E” - Only “E” is really meaningful.
Returns: complex | float
Raises: -

function em.MWField.**boundary()**

Interpolates the field solution on the boundary specified by a surface. The E-field will be computed on the boundary triangle vertices.

The triangulation and data structure is passed to the EHField object allowing it to be used with the [add_field](#) function of the display class.

Parameters • selection: FaceSelection | GeoSurface - The boundary to compute the E-field on.
Returns: EHField
Raises: -

function em.MWField.**current_boundary()**

Interpolates the field solution on the boundary specified by a surface. The E-field will be computed on the boundary triangle vertices. This function also computes the current density on the surface which is otherwise not included when calling [boundary](#).

The triangulation and data structure is passed to the EHField object allowing it to be used with the [add_field](#) function of the display class.

Parameters • selection: FaceSelection | GeoSurface - The boundary to compute the E-field on.
Returns: EHField
Raises: -

function em.MWField.**cutplane()**

Computes the coordinates for a cut-plane through the simulation domain based on a specified grid step size. Then returns an EHField object (see [EHField](#)) with the field data and meta data.

The cutplane is either an XY plane, YZ plane or XZ plane.

Parameters • ds: float - The grid step size
• x: float | None = None - If provided an YZ-plane is generated at the given x-coordinate.
• y: float | None = None - If provided an XZ-plane is generated at the given y-coordinate.
• z: float | None = None - If provided an XY-plane is generated at the given z-coordinate.
Returns: EHField
Raises: -

function em.MWField.cutplane_normal()

Computes the coordinates for a cut-plane through the simulation domain based on a specified grid step size. Then returns an EHField object (see EHField) with the field data and meta data.

This cutplane is defined in any arbitrary orientation.

- Parameters**
- point: tuple[float, float, float] - The plane normal axis origin.
 - normal: tuple[float, float, float] - The plane normal vector.
 - npoints: int = 300 - The number of points along each plane vector.
 - usenon: bool = True - If points outside the simulation domain should be NaN instead of 0.0.

Returns: EHField

Raises: -

function em.MWField.grid()

Evaluates a 3D grid enclosing the entire simulation domain (or a restriction) and returns an EHField object with the required metadata.

You can either specify a grid step size or a total number of points in the 3D grid.

- Parameters**
- ds: float | None = None - The grid step size in meters
 - N: int = 10_000, The number of total grid points. Not exactly respected.
 - usenon: bool = True - If points outside the simulation domain should contain NaN instead of 0s.
 - x_range: tuple[float, float] | None = None - An x-range restriction. Otherwise it uses the geometry bounds.
 - y_range: tuple[float, float] | None = None - An z-range restriction. Otherwise it uses the geometry bounds.
 - z_range: tuple[float, float] | None = None - An z-range restriction. Otherwise it uses the geometry bounds.

Returns: EHField

Raises: -

function em.MWField.farfield_2d()

Computes the farfield radiation pattern in a single polar cutplane using the Stratton-Chu integrals and returns the data in an EHFieldFF object (see EHFieldFF).

- Parameters**
- ref_direction: tuple[float, float, float] | Axis - The reference direction equivalent to the zero-angle in the local polar coordinates.
 - plane_normal: tuple[float, float, float] | Axis - The plane normal for the local 2D polar coordinates.
 - faces: FaceSelection | GeoSurface - The surface to integrate the E and H-field over.
 - ang_range: tuple[float, float] = (-180, 180) - The angular range in degrees.
 - Npoints: int = 201 - The total number of points in the 2D polar coordinates
 - origin: tuple[float, float, float] - An optional radiation origin to use to compute an outward normal vector in cases where from the geometry it is not easy for EMerge to infer what is to be consider inside and outward pointing.
 - syms: 'Ex' | 'Ey' | 'Ez' | 'Hx' | 'Hy' | 'Hz' | None = The simulation symmetries to use. 'Ez' would correspond to a PEC symmetry in the XY plane. Hy would correspond to a PMC symmetry in the XZ plane. etc.

Returns: EHFieldFF

Raises: -

function em.MWField.farfield_3d()

Computes the farfield radiation pattern in a spherical 2D surface (θ, φ) using the Stratton-Chu integrals and returns the data in an EHFieldFF object (see EHFieldFF).

- Parameters**
- faces: FaceSelection | GeoSurface - The surface to integrate the E and H-field over.
 - thetas: np.ndarray | None - The theta angle axis in global coordinates. Defaults to (0, π) in 91 steps.
 - phis: np.ndarray | None - The phi angle axis in global coordinates. Defaults to ($-\pi, \pi$) in 181 steps
 - origin: tuple[float, float, float] - An optional radiation origin to use to compute an outward normal vector in cases where from the geometry it is not easy for EMerge to infer what is to be consider inside and outward pointing.
 - syms: 'Ex' | 'Ey' | 'Ez' | 'Hx' | 'Hy' | 'Hz' | None = The simulation symmetries to use. 'Ez' would correspond to a PEC symmetry in the XY plane. Hy would correspond to a PMC symmetry in the XZ plane. etc.

Returns: EHFieldFF

Raises: -

function em.MWField.farfield()

Computes the farfield radiation pattern on the requested pherical coordinates using the Stratton-Chu integrals and returns the data in an EHFieldFF object (see EHFieldFF).

- Parameters**
- thetas: np.ndarray - The theta angle axis in global coordinates.
 - phis: np.ndarray | None - The phi angle axis in global coordinates.
 - faces: FaceSelection | GeoSurface - The surface to integrate the E and H-field over.
 - origin: tuple[float, float, float] - An optional radiation origin to use to compute an outward normal vector in cases where from the geometry it is not easy for EMerge to infer what is to be consider inside and outward pointing.
 - syms: 'Ex' | 'Ey' | 'Ez' | 'Hx' | 'Hy' | 'Hz' | None = The simulation symmetries to use. 'Ez' would correspond to a PEC symmetry in the XY plane. Hy would correspond to a PMC symmetry in the XZ plane. etc.

Returns: EHFieldFF

Raises: -

10.1 EHField class

The EHField class contains all the different possible electric and magnetic fields (E,H,D,B,P,S,Sm,J,Q) and the coordinates where they where computed. It also contains metadata on the structure of the underlying coordinates (2D grid, 3D grid, unstructured grid, triangulated surface etc.).

Technicall the EHfield and EHFieldFF classes are not part of EMerge but part of the shared `emsutil` library which is also used by Optycal.

All field attributes are: E, H, S, P, D, B, Ex, Ey, Ez, Hx, ... etc, Sm (time averaged Poynting vector), Js, Jv, normE, normB, normS, normSm etc.

function em.EHField()

A data container class containing the complete set of different fields at specified coordinates. Also contains metadata regarding the underlying structure of the coordinates.

Parameters Should not be created manually

Returns: EHField

Raises: -

function em.EHField.vector()

Returns a FieldPlotData object to be used with the `add_field` function when creating a vector plot.

- Parameters**
- field: "E" | "H" | "B" | "D" | "P" | "S" | "Sm" | "Jv" - The field to plot
 - metric: "real" | "imag" | "complex" - Which metric of the complex field vlaue to plot.

Returns: FieldPlotData

Raises: -

function em.EHField.**scalar()**

Returns a FieldPlotData object to be used with the `add_field` function when creating a scalar plot.

Parameters	- field: “Ex” “Ey” “Hx” “normE” etc. - The field to plot - metric: “real” “imag” “complex” “abs” - Which metric of the complex field vlaue to plot.
Returns:	FieldPlotData
Raises:	-

The EHFieldFF class contains the following farfield properties: E, H, Ex, Ey, Ez, Hx, Hy, Hz, Etheta, Ephi, Htheta, Hphi, Erhcp, Elhcp, AR, gain, dir, RCS.

Some attributes allow for further sub attribute specifications as they return an object of type `FarFieldComponent`. This class itself has the attributes: x, y, z, theta, phi, lhcp, rhcp and norm.

function em.**FarFieldComponent()**

A data container class for farfield quantities.

Parameters	Should not be created by users.
Returns:	FarFieldComponent
Raises:	-

function em.**EHFieldFF()**

A data container class containing the complete set of different farfield components at specified coordinates. Also contains metadata regarding the underlaying structure of the coordinates.

Parameters	Should not be created by users.
Returns:	EHFieldFF
Raises:	-

property em.EHFieldFF.**ang**

The local 1D polar coordinate angle array. In arbitrary polar coordinate slices, each angle is represented by a pair of θ and φ coordinates. This ang axis always contains the angles relative to the local polar coordinate plane as defined by `farfield_2d`,

Returns:	np.ndarray
-----------------	------------

function em.EHFieldFF.**total_radiated_power_integral()**

Performs a power integral assuming that the farfield angles are a complete closed 2D set of points along the surface.

Parameters	- r: float = 1.0 - The numerical radius to compute the farfiel dat. Does not impact the solution. - use: “EH” - Which field definition to use. Defaults to “EH” only option.
Returns:	FieldPlotData
Raises:	-

property em.EHFieldFF.**E**

Returns the E-farfield component. Can be used as following:

<pre>1 field.farfield_3d().E.lhcp</pre>
Returns: FarFieldComponent

property em.EHFieldFF.**H**

Returns the H-farfield component. Can be used as following:

<pre>1 field.farfield_3d().H.norm</pre>
Returns: FarFieldComponent

property em.EHFieldFF.**gain**

Returns the isotropic gain farfield component.

<pre>1 field.farfield_3d().gain.theta</pre>
Returns: FarFieldComponent

property em.EHFieldFF.**dir**

Returns the directivity farfield component.

<pre>1 field.farfield_3d().dir.theta</pre>
Returns: FarFieldComponent

function em.EHFieldFF.**surfplot()**

Returns the plot data (FieldPlotData) to pass to the the `add_field` function to plot farfield plots in 3D.

Parameters	- polarization: str - A string that retrieves the farfield component to be plotted. You can use this as if you are selecting attributes. Example: <code>E</code> or <code>gain.lhcp</code> etc. - quantity: “abs” “real” “imag” “angle” - Which farfield component quantity to plot. - dB: bool = False - Converts the quanity to decibels. - dBfloor: float = -30.0 - The dB level to correspond to a o-radius. - rmax: float None - The maximum radius to use for the greatest farfield quantity. - offset: tuple[float, float, float] = (0,0,0) - The 3D origin for the farfield plot.
Returns:	FieldPlotData
Raises:	-

11 Display

The following functions can be called on the PVDisplay class which is the PyVista interface class available under:

<pre>1 my_model.display.</pre>

function PVDisplay.**add_anchors()**

Add a series of achor points to the plot window. See `Anchor`

Parameters	anchors: list[Anchor] - A list of anchor objects size: float = 1.0 - A sizing scaler to control how big the anchors are plotted
Returns:	None
Raises:	-

function PVDisplay.**add_object()**

Adds an EMerge GeoObject or Selection of any dimension to the plot window.

Parameters	- obj: GeoObject Selection - The object or selection to add - mesh: bool = False - If the mesh of the object should be added instead of a 3D render of the geometry - volume_mesh: bool = True - If true, the volume mesh is plotted, otherwise only the surface meshes. - label: bool = False - If a label is to be added with the object/selectio name - label_text: str None = None - What text to add in case a label is added - texture: str None = None - Specify a texture name. Used when automatically plotting boundary conditions. - opacity: float None = None - The opacity of the object/selection. Will take the lowest of this value and the objects own opacity - draw_line: bool = True - If true, thicker lines outlining the geometry are plotted - pbr: bool = True - If physics based rendering in PyVista should be allowed. - smooth_shading: bool = False - If smooth shading should be used during physics based rendering. This renders spherical/curved geometries better but the corners for boxes will look rounded. - selectable_as: str None = None - Makes the object selectable under a specific name.
Returns:	None
Raises:	-

function PVDisplay.add_objects()

Add multiple objects with a certain opacity as a list of objects. Also accepts all `add_object` arguments.

- Parameters
- `*objects`: `GeoObject` | `Selection` - A sequence of object arguments
 - `opacity`: `float` | `None` = `None` - A maximum opacity for the objects
 - `**kwargs`: Other keyword arguments matching `add_object`.
- Returns: `None`
- Raises: -

function PVDisplay.populate()

Add all geometries in the current model to the plot.

- Parameters
- `opacity`: `float` | `None` = `None` - A maximum opacity for all objects.
 - `**kwargs` - Other keyword arguments matching `add_object`.
- Returns: `None`
- Raises: -

function PVDisplay.add_scatter()

Add scatter plot points to the 3D plot window.

- Parameters
- `xs`: `np.ndarray` - A numpy array of x-coordinates.
 - `ys`: `np.ndarray` - A numpy array of y-coordinates.
 - `zs`: `np.ndarray` - A numpy array of z-coordinates.
- Returns: `None`
- Raises: -

function PVDisplay.add_line()

Add an EMerge Line object to the plot. See `Line` for more details.

- Parameters
- `line`: `Line` - The line object
 - `width`: `float` = `3.0` - The line width
 - `color`: `str` = “EMERGE-RED” - The line color
- Returns: `None`
- Raises: -

function PVDisplay.add_portmode()

Add the port mode of a Port boundary condition object (any of the port boundary conditions) to the 3D plot.

- Parameters
- `port`: `PortBC` - The Port Boundary Condition
 - `dv`: `tuple[float, float, float]` = `(0,0,0)` - A slight offset to give the port mode plot elements. Sometimes useful if the port mode is z-fighting with a nother plot.
 - `XYZ`: `tuple[np.ndarray,...]` - A set coordinates where to evaluate the port mode quiver plot. By default it will pick triangle centers.
 - `field`: “E” | “H” | “gradE” | “Ez” - Which port mode field componet to plot.
 - `ko`: `float` | `None` = `None` - At which simulation phase constant to plot the mode. quasi-TEM modes can change shape.
 - `cmap`: `str` | `None` = `None` - A colormap to use for the quiver plot
 - `mode_number`: `int` | `None` = `None` - A selection of which mode number to plot. Default to the first mode
 - `color`: `str` | `None` = `None` - A color string or hex code to use for the arrows.
 - `Npoints`: Deprecated
- Returns: `None`
- Raises: -

function PVDisplay.add_farfield3d()

Add the results of a 3D farfield calculation to the plot window as a 3D farfield pattern.

The Farfield data objects can be generated as following:

python

```
1 solution = model.mw.run_sweep()
2 field = solution.field.find(freq=...)
3 ff3d = field.farfield_3d(ff_boundary)
```

Choices for the farfield component are: `Ex`, `Ey`, `Ez`, `Hx`, `Hy`, `Hx`, `Sx`, `Sy`, `Sz`, `Etheta`, `Ephi`, `Htheta`, `Hphi`, `Stheta`, `Sphi`, `Elhcp`, `Erhcp`, `AR`, `gain.norm`. `gain.Ex`, `gain.Ey`, `gain.Ez`, `gain.Etheta`, `gain.Ephi`, `gain.Erhcp`, `gain.Elhcp`, `normE`, `normH`, `normS`.

- Parameters
- `farfield_obj`: `EHFieldFF` - The farfield data objects.
 - `component`: “Ex”, “Elhcp”, “Hx”, “Htheta”, “gain.norm” = “normE” - The field component to plot.
 - `quantity`: “abs” | “real” | “imag” | “angle” - Which quantity of the quantity to plot. Defaults to “abs”
 - `dB`: `bool` = `False` - If the field should be converted to dB.
 - `dBfloor`: `float` = `-30`. Which dB level to use for the 3D polar plot `R=0`.
 - `rmax`: `float` | `None` = `None` - Which maximum radius in meters to use for the highest farfield plot magnitude.
 - `offset`: `tuple[float, float, float]` - Where to put the farfield plot. Defaults to the origin.
 - `opacity`: `float` | `None` = `None` - The opacity of the farfield plot.
- Returns: `None`
- Raises: -

function EMergeDisplay.cbar()

Method chaining function to specify the color bar details for the plot that will be added next. Example usage:

python

```
1 model.display.cbar(...).add_field()
```

Multiple plot with the same color bar name will share the same color bar. Final settings apply.

- Parameters
- `name`: `str` - The name for the color bar.
 - `n_labels`: `int` = `5` - How many value labels to add
 - `interactive`: `bool` = `False` -
 - `clim`: `tuple[float, float]` - The color bar limits lowest and highest value.
- Returns: `EMergeDisplay`
- Raises: -

function EMergeDisplay.set_theme()

Apply a color theme as `EMergeTheme` object.

- Parameters
- `theme`: `EMergeTheme` - The `EMergeTheme` object.
- Returns: `None`
- Raises: -

function EMergeDisplay.show()

Tells the display to show generating a new window. May not get priority from the operating system.

- Parameters
- `screenshot`: `str` | `None` = `None` - If a pathname is provided a screenshot will be automatically saved.
 - `off_screen`: `bool` = `False` - Keeps the window off screen. Useful if you only want to create a screenshot.
- Returns: `None`
- Raises: -

function EMergeDisplay.**animate()**

A method chaining function to tell the plot window that the next plot function should be animated (if possible).

Make sure that the field component you plot is “complex”. Otherwise all waves will be standing waves.

Example usage:

python

```
1 model.display.animate().add_field()
```

Parameters

- Nsteps: int = 35 - The number of steps to animate for a complete 360 degree phase cycle.
- fps: int = 25 - The number of frames per second for the animation.

Returns: EMergeDisplay

Raises: -

function EMergeDisplay.**drawing_bw()**

Sets the drawing mode to black and white.

Parameters

Returns: EMergeDisplay

Raises: -

function EMergeDisplay.**save_vtk()**

Saves the current plot window elements to a VTK file.

Parameters base_path: str - The path to store the file to.

Returns: None

Raises: -

function EMergeDisplay.**add_scatter()**

Add scatter plot points to the 3D plot window.

Parameters

- xs: np.ndarray - A numpy array of x-coordinates.
- ys: np.ndarray - A numpy array of y-coordinates.
- zs: np.ndarray - A numpy array of z-coordinates.

Returns: None

Raises: -

function PVDDisplay.**add_field()**

Add an generid FieldPlotData object to the plot window. These are created from datasets and contain all metadata to tell the display class how to display it.

Example of FieldPlotData creation:

python

```
1 solution = sim.mw.run_sweep()
2 fielddata = solution.field.find(freq=1e9).grid(N=100_000)
```

Parameters

- field: FieldPlotData - The field data object
- scale: “lin” | “log” | “symlog” = “lin” - A resaling to apply. Lin is default (no scaling) log applies a logarithm and symlog applies a symmetric logarithm for more detail in wave animations.
- cmap: str | None - The colormap name to use
- clim: tuple[float, float] - The color bar limits to use.
- opacity: float | None = None - The opacity of the field.
- voltype: “cloud” | “contour” | “clip” - Which plot style to use for volumetric data (3D data arrays). “cloud” is a cloud plot. “contour” is a 3D contour (NLevels sets the number of contour isosurfaces), “clip” adds a cloud like plot with a movable clip plane.
- clim_crop_factor: float = 1.0 - A scaling factor for the color bar limits if defaults are chosen. 0.5 means that the color bar will reach to half the maximum detected value.
- symmetrize: bool = False - If true, the colorbar will always range from -vmax to +vmax where vmax is max(abs(values)). Will also automatically select a symmetric colormap if no value is provided.
- _fieldname: str - The name to use for the plotted field. Defaults to a name selected by the field data.

Returns: None

Raises: -

function EMergeDisplay.**add_surf()**

Add a 2D surface plot to the plot window.

Parameters

- x: np.ndarray - a (N,M) array of X-coordinates.
- y: np.ndarray - a (N,M) array of Y-coordinates.
- z: np.ndarray - a (N,M) array of z-coordinates
- field: np.ndarray - a (N,M) field of scalar values for each coordinate.
- scale: “lin” | “log” | “symlog” = “lin” - A resaling to apply. Lin is default (no scaling) log applies a logarithm and symlog applies a symmetric logarithm for more detail in wave animations.
- cmap: str | None - The colormap name to use.
- clim: tuple[float, float] - The color bar limits to use.
 - opacity: float | None = None - The opacity of the field.
 - clim_crop_factor: float = 1.0 - A scaling factor for the color bar limits if defaults are chosen. 0.5 means that the color bar will reach to half the maximum detected value.
 - symmetrize: bool = False - If true, the colorbar will always range from -vmax to +vmax where vmax is max(abs(values)). Will also automatically select a symmetric colormap if no value is provided.
 - _fieldname: str - The name to use for the plotted field. Defaults to a name selected by the field data.

Returns: None

Raises: -

function EMergeDisplay.**add_trisurf()**

Adds an unstructured triangulated surface plot.

Parameters

- x: np.ndarray - a (N,) array of X-coordinates.
- y: np.ndarray - a (N,) array of Y-coordinates.
- z: np.ndarray - a (N,) array of z-coordinates
- field: np.ndarray - a (N,) field of scalar values for each coordinate.
- tris: np.ndarray - a (3,M) array for the triangulation of the point set.
- scale: “lin” | “log” | “symlog” = “lin” - A resaling to apply. Lin is default (no scaling) log applies a logarithm and symlog applies a symmetric logarithm for more detail in wave animations.
- cmap: str | None - The colormap name to use.
- clim: tuple[float, float] - The color bar limits to use.
 - opacity: float | None = None - The opacity of the field.
 - clim_crop_factor: float = 1.0 - A scaling factor for the color bar limits if defaults are chosen. 0.5 means that the color bar will reach to half the maximum detected value.
 - symmetrize: bool = False - If true, the colorbar will always range from -vmax to +vmax where vmax is max(abs(values)). Will also automatically select a symmetric colormap if no value is provided.
 - _fieldname: str - The name to use for the plotted field. Defaults to a name selected by the field data.

Returns: None

Raises: -

function EMergeDisplay.add_clip_volume()

Adds a 3D clip volume plot using 3D strutured data.

- Parameters**
- X: np.ndarray - a (M,N,K) array of X-coordinates.
 - X: np.ndarray - a (M,N,K) array of Y-coordinates.
 - X: np.ndarray - a (M,N,K) array of z-coordinates
 - V: np.ndarray - a (M,N,K) field of scalar values for each coordinate.
 - scale: “lin” | “log” | “symlog” = “lin” - A resaling to apply. Lin is default (no scaling) log applies a logarithm and symlog applies a symmetric logarithm for more detail in wave animations.
 - cmap: str | None - The colormap name to use.
 - clim: tuple[float, float] - The color bar limits to use.
 - opacity: float | None = None - The opacity of the field.
 - clim_crop_factor: float = 1.0 - A scaling factor for the color bar limits if defaults are chosen. 0.5 means that the color bar will reach to half the maximum detected value.
 - symmetrize: bool = False - If true, the colorbar will always range from -vmax to +vmax where vmax is max(abs(values)). Will also automatically select a symmetric colormap if no value is provided.
 - _fieldname: str - The name to use for the plotted field. Defaults to a name selected by the field data.

Returns: None

Raises: -

function EMergeDisplay.add_contour()

Adds a 3D contour volume plot using 3D strutured data.

- Parameters**
- X: np.ndarray - a (M,N,K) array of X-coordinates.
 - X: np.ndarray - a (M,N,K) array of Y-coordinates.
 - X: np.ndarray - a (M,N,K) array of z-coordinates
 - V: np.ndarray - a (M,N,K) field of scalar values for each coordinate.
 - Nlevels: int = 5 - The number of contour plot isosurface levels.
 - scale: “lin” | “log” | “symlog” = “lin” - A resaling to apply. Lin is default (no scaling) log applies a logarithm and symlog applies a symmetric logarithm for more detail in wave animations.
 - cmap: str | None - The colormap name to use.
 - clim: tuple[float, float] - The color bar limits to use.
 - opacity: float | None = None - The opacity of the field.
 - clim_crop_factor: float = 1.0 - A scaling factor for the color bar limits if defaults are chosen. 0.5 means that the color bar will reach to half the maximum detected value.
 - symmetrize: bool = False - If true, the colorbar will always range from -vmax to +vmax where vmax is max(abs(values)). Will also automatically select a symmetric colormap if no value is provided.
 - _fieldname: str - The name to use for the plotted field. Defaults to a name selected by the field data.

Returns: None

Raises: -

function EMergeDisplay.add_cloud()

Adds a 3D cloud plot using 3D strutured data.

- Parameters**
- X: np.ndarray - a (M,N,K) array of X-coordinates.
 - X: np.ndarray - a (M,N,K) array of Y-coordinates.
 - X: np.ndarray - a (M,N,K) array of z-coordinates
 - V: np.ndarray - a (M,N,K) field of scalar values for each coordinate.
 - scale: “lin” | “log” | “symlog” = “lin” - A resaling to apply. Lin is default (no scaling) log applies a logarithm and symlog applies a symmetric logarithm for more detail in wave animations.
 - cmap: str | None - The colormap name to use.
 - clim: tuple[float, float] - The color bar limits to use.
 - opacity: float | None = None - The opacity of the field.
 - clim_crop_factor: float = 1.0 - A scaling factor for the color bar limits if defaults are chosen. 0.5 means that the color bar will reach to half the maximum detected value.
 - symmetrize: bool = False - If true, the colorbar will always range from -vmax to +vmax where vmax is max(abs(values)). Will also automatically select a symmetric colormap if no value is provided.
 - _fieldname: str - The name to use for the plotted field. Defaults to a name selected by the field data.

Returns: None

Raises: -

function EMergeDisplay.add_quiver()

Adds a quiver vector arrow plot to the plot window.

- Parameters**
- x: np.ndarray - a (N,) array of X-coordinates.
 - y: np.ndarray - a (N,) array of Y-coordinates.
 - z: np.ndarray - a (N,) array of z-coordinates.
 - dx: np.ndarray - a (N,) array of X-vector components.
 - dy: np.ndarray - a (N,) array of Y-vector components.
 - dz: np.ndarray - a (N,) array of z-vector components.
 - scale: float = 1.0 - An extra size scaling to apply over the vector lengths.
 - scalemode: “lin” | “log” = “lin” - A resaling to apply. Lin is default (no scaling) log applies a logarithm.
 - cmap: str | None - The colormap name to use.
 - clim: tuple[float, float] - The color bar limits to use.
 - opacity: float | None = None - The opacity of the field.
 - _fieldname: str - The name to use for the plotted field. Defaults to a name selected by the field data.

Returns: None

Raises: -

function EMergeDisplay.add_streamline()

Adds a streamline plot to the plot window using vector field like data.

- Parameters**
- x: np.ndarray - a (N,) array of X-coordinates.
 - y: np.ndarray - a (N,) array of Y-coordinates.
 - z: np.ndarray - a (N,) array of z-coordinates.
 - dx: np.ndarray - a (N,) array of X-vector components.
 - dy: np.ndarray - a (N,) array of Y-vector components.
 - dz: np.ndarray - a (N,) array of z-vector components.=
 - source_points: np.ndarray | None = None - A (3,M) array of source point coordinates where to start streamlines from.
 - scale: float = 1.0 - An extra size scaling to apply over the vector lengths.
 - scalemode: “lin” | “log” = “lin” - A resaling to apply. Lin is default (no scaling) log applies a logarithm.
 - cmap: str | None - The colormap name to use.
 - clim: tuple[float, float] - The color bar limits to use.
 - opacity: float | None = None - The opacity of the field.
 - _fieldname: str - The name to use for the plotted field. Defaults to a name selected by the field data.

Returns: None

Raises: -

function EMergeDisplay.add_title()

Adds a tile text to the plot

- Parameters**
- title: str - The title string
 - color: str - The color to use for the text.

Returns: None

Raises: -

function EMergeDisplay.add_text()

Adds text to the plot window

- Parameters**
- text: str - The text string
 - color: str - The color to use for the text.
 - position: “lower_left” | “lower_right” | “upper_left” | “upper_right” | “lower_edge” | “upper_edge” | “right_edge” | “left_edge” - The position of the text.
 - abs_position: tuple[float, float] | None - The absolute position in the viewport.

Returns: None

Raises: -